Lecture 10. Reinforcement learning

Value functions, Bellman equations, policy gradients and deep networks

Deep Learning for Scientific Discovery

2026/2027

1 The reinforcement-learning setting

A rover on the surface of Mars must drive from its landing site to a site where it can collect a rock sample. The terrain is divided into square tiles, a crater lies near the sample site, and the sandy ground occasionally makes the rover slip sideways. At every step the rover knows on which tile it stands and chooses one of four directions to drive. No map of good moves is provided. The rover learns only from what happens, a reward when it collects the sample and a penalty when it falls into the crater. A player of the board game Go faces the same kind of problem on a much larger scale. At every turn the player sees the position on the board, chooses a move, the opponent answers, and only at the end of the game is it known whether the moves led to a win or a loss.

The models of supervised learning, from a network that classifies images to a random forest that predicts a property from a table of measured quantities, learn from a data set that exists before training. Every input carries its correct label, and the data do not change while the model learns. An agent in reinforcement learning (RL) receives no labels and starts without a data set. It acts, and its actions decide which situations it meets and therefore which data it learns from [1]. Nothing indicates to the rover which direction is best on a given tile, and for a Go position the best move is unknown even to experts. A program trained to imitate recorded games of strong players is limited by their play, while a program that learns from the results of its own games has no such limit. The feedback is a number that rates an outcome, and it does not say what should have been done instead. This feedback is often delayed. The rover receives nothing until it reaches the sample site or the crater, and a Go player learns the result only after hundreds of moves. Which of the earlier decisions were responsible must be inferred, a difficulty called the *credit assignment problem*. Since the learner produces its own data by acting, a poor strategy experiences only the situations that a poor strategy leads to, and the data improve only as the strategy improves.

Many problems in science share this structure, a sequence of decisions whose quality shows only in a later outcome. The voltages of the magnetic coils of a tokamak must be adjusted continuously to hold the plasma of a fusion experiment in a prescribed shape [2]. An algorithm for multiplying matrices is a sequence of elementary operations, and the search for one with fewer multiplications can be posed as a game [3]. The treatment of a patient in intensive care is a sequence of doses whose effect becomes clear only at the end of the stay [4]. The pulses applied to a quantum device must bring it to a target state despite imperfect control and noisy measurements [5]. Section 10 returns to these examples.

Reinforcement learning maximises an expected total reward, but it differs from the optimisation of a known function in three ways. The objective can only be sampled by acting, its gradient with respect to the strategy has no explicit expression, and the distribution of the

data changes as the strategy improves.

Sections 2 and 3 introduce the vocabulary. Section 4 defines what a good strategy is through the value of a state, the expected return collected from it, where the return is the sum of the future rewards with a weight that decreases with their distance in time. The values obey the Bellman equations (Section 5). When the dynamics of the environment are known, these equations are solved exactly, by a linear system for a given strategy and by value iteration for the best one (Section 6). When the dynamics are unknown, temporal-difference learning and Q-learning perform the same iteration with sampled transitions in place of averages (Section 7). The policy gradient of Section 8 takes the other route and adjusts the strategy itself in the direction that raises the expected return. Section 9 shows where deep networks enter, as replacements for the tables of values and of action probabilities when the states are continuous or too many, and Section 10 describes applications in science. The rover problem is small enough to be solved exactly, so every learning method can be checked against the true answer, and two simulated control tasks, landing a spacecraft and balancing a pole, demonstrate the methods with neural networks.

2 Agent, environment and reward

Each concept of this and the next section is introduced with its symbol and its meaning for the rover, in the order of [1], and Table 1 collects them.

2.1 Agent and environment

The learner and decision maker is the *agent*, here the rover or the Go player. Everything it interacts with is the *environment*, the terrain or the board together with the opponent. Time advances in discrete steps $t = 0, 1, 2, \ldots$. At step t the agent observes the state s_t of the environment and chooses an action a_t. The action changes the state to s_{t+1}, and the environment returns a single number, the reward r_{t+1} (Figure 1, left). The agent's rule for choosing actions is its policy $\pi(a \mid s)$, and the environment's rule for answering is a probability p of the next state and reward. Both are defined in Section 3.

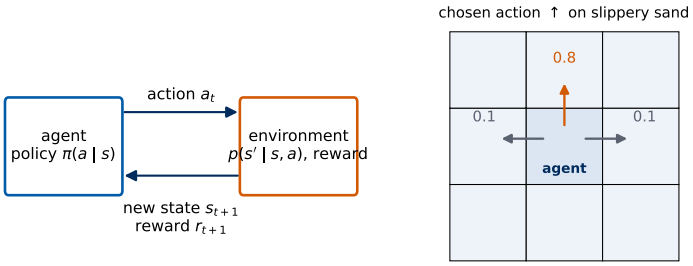


Figure 1: The left panel shows the interaction loop of agent and environment. The right panel shows one tile of the terrain when the rover chooses ↑. It moves up with probability 0.8 and slips left or right with probability 0.1 each.

2.2 The rover terrain

The rover is the running example of the chapter. It is a gridworld, an environment small enough that all its quantities can be computed and drawn as maps. The rover drives on a terrain of 5×5 square tiles (Figure 2, left). Rows are numbered 0 to 4 from the top and

columns 0 to 4 from the left, and a tile is written (row, column). A boulder occupies the centre tile (2,2) and cannot be entered. The landing site S is the bottom-left tile (4,0). The sample site X lies at (0,4) and the crater C at (1,3), directly left of the tile below X and directly below the tile left of X. The task is to drive from S to X in few steps without falling into C. One run of the rover from S until it reaches X or C is an *episode*, and every episode starts again at S. The simplest rule for driving is the *random policy*, which chooses each of the four directions with probability 1/4 on every tile, whatever happened before. The rover does not know the map or the slip probabilities. It sees only its tile and the reward after each step. The environment is a simulation defined by the rules of this subsection and the next two, and we know its dynamics because we wrote them down.

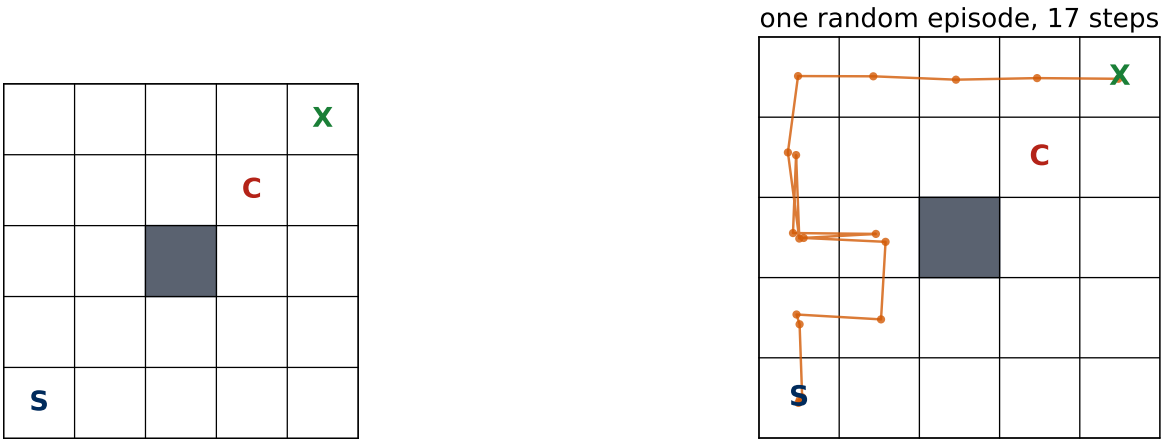


Figure 2: The left panel shows the terrain. S landing site (4,0), X sample site (0,4) with reward +10, C crater (1,3) with reward −10, grey boulder (2,2). The right panel shows one episode of the random policy, which chooses each of the four directions with probability 1/4. It lasts 17 steps, and every reward is zero except the last one, +10 on entering X.

2.3 States and actions

The *state space* $\mathcal{S}$ is the set of all states the environment can be in. For the rover the state is the tile on which it stands, and with the boulder excluded there are 24 states. For Go it is the position on the board, and for a spacecraft its position, velocity and angle. The *action space* $\mathcal{A}$ is the set of all actions the agent can take. The rover chooses one of the four steps $\uparrow, \downarrow, \leftarrow, \rightarrow$ on every tile, a Go player chooses a legal move. The sand is slippery, so the rover moves in the chosen direction with probability 0.8 and slips to each of the two perpendicular directions with probability 0.1 (Figure 1, right). A move into the boulder or off the terrain leaves the rover on its tile. When the agent receives only part of the state, it receives an *observation*, and the problem is called partially observed. The rover observes its full state.

2.4 The reward and the goal

After every action the environment returns the reward $r_{t+1} \in \mathbb{R}$. It carries the index $t+1$ because it arrives together with the new state s_{t+1}. The rover receives +10 for entering X, −10 for entering C and 0 for every other step. Both X and C end the episode and are called *terminal states*. In Go the reward is +1 for a won game, −1 for a lost one, and 0 for every move before the end. The reward rates what has just happened and does not say which action would have been right. It defines the task, and an agent maximises what is rewarded and

nothing else.

The interaction produces the sequence $s_0, a_0, r_1, s_1, a_1, r_2, s_2, \ldots$, called a *trajectory*, which lists the states, the actions and the rewards. An episode is the trajectory from the start to a terminal state. Its final time T is the step at which the terminal state is entered, and it is random, since slips and replies of the opponent change the length of the episode. The goal is to choose actions that make the undiscounted total reward of an episode,

$$R_{\text{tot}} := r_1 + r_2 + \cdots + r_T, \tag{1}$$

as large as possible on average over the slips of the ground or the replies of the opponent. Maximising the next reward alone is not enough, because a step without reward, such as a step towards the sample site or a quiet move in Go, can be necessary for a large reward later. And a short path can end in the crater.

2.5 The policy

The agent chooses its actions according to a *policy*, a rule that assigns to every state a probability for each action. A deterministic policy gives one action per state. Figure 3 shows two deterministic policies on the terrain. The policy $\pi_\uparrow$ chooses $\uparrow$ on every tile outside the top row and $\rightarrow$ in the top row. From S it climbs the first column and then drives along the top row to X. The policy $\pi_\rightarrow$ chooses $\rightarrow$ on every tile outside the last column and $\uparrow$ in the last column. From S it drives along the bottom row and then climbs the last column, passing the crater on its left. The two policies are mirror images under reflection on the diagonal through S and X. Reinforcement learning is the search for a policy with a large total reward.

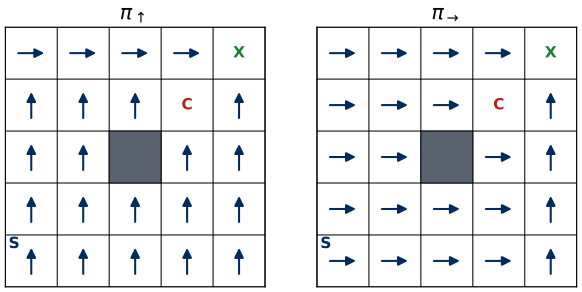


Figure 3: The two hand-made policies, $\pi_\uparrow$ (left) and $\pi_\rightarrow$ (right), shown by one arrow per non-terminal tile.

The right panel of Figure 2 shows what an agent experiences under the random policy. It moves erratically, runs into the edges, slips sideways and, in this episode, reaches X after 17 steps. The record of the episode is a list of transitions $(s_t, a_t, r_{t+1}, s_{t+1})$, all with reward zero except the last, so $R_{\text{tot}} = 10$. Which of the earlier moves were good is not recorded anywhere. A good policy reaches X in eight steps, also with $R_{\text{tot}} = 10$, so the plain sum (1) does not prefer the faster agent. The discount of Section 3.4 removes this defect.

3 Markov decision processes

3.1 The Markov property

The total reward depends on future rewards, and these depend on how the environment responds to the agent's actions. To reason about it we need a model of this response. The

standard model rests on an *assumption*, the Markov property, which states that the next state and the next reward depend on the history only through the current state and action,

$$p(s_{t+1}, r_{t+1} \mid s_0, a_0, \ldots, s_t, a_t) = p(s_{t+1}, r_{t+1} \mid s_t, a_t). \tag{2}$$

The state is then a sufficient statistic for the future. The position of a thrown ball alone does not predict its next position, while position and velocity together do. Whether a description is Markov depends on what we include in the state. If the environment has memory, the property is restored by adding to the state whatever must be remembered. For a partially observed problem a common remedy is to feed the recent history of observations to the agent, for example through a recurrent network. The tile of the rover is a Markov state, because the slip probabilities depend only on the tile and the chosen direction.

3.2 The dynamics of the environment

Under the Markov assumption the environment is described by a single object, the *dynamics*

$$p(s', r \mid s, a) \coloneqq p(s_{t+1} = s',\, r_{t+1} = r \mid s_t = s,\, a_t = a). \tag{3}$$

It answers the question where the agent lands and which reward it receives when it takes action a in state s. The dynamics are the full description of the environment, its *model*. For a deterministic environment p is 0 or 1. On the tile $s = (1, 4)$ directly below X and for the action $\uparrow$, the rover enters X and receives $+10$ with probability 0.8, slips to the left into C and receives -10 with probability 0.1, and slips to the right, off the terrain, staying on its tile with reward 0, with probability 0.1. Hence

$$p(\mathrm{X}, +10 \mid s, \uparrow) = 0.8, \qquad p(\mathrm{C}, -10 \mid s, \uparrow) = 0.1, \qquad p(s, 0 \mid s, \uparrow) = 0.1, \tag{4}$$

and every other pair (s', r) has probability zero.

Every other description of the environment follows from $p(s', r \mid s, a)$ by summation. Summing over the rewards gives the *state-transition probability*

$$p(s' \mid s, a) \coloneqq \sum_r p(s', r \mid s, a), \tag{5}$$

the probability of landing in s' whatever the reward. On the tile below X it gives $p(\mathrm{X} \mid s, \uparrow) = 0.8$, $p(\mathrm{C} \mid s, \uparrow) = 0.1$ and $p(s \mid s, \uparrow) = 0.1$. Averaging the reward over both outcomes gives the *expected reward* of a state–action pair,

$$r(s, a) \coloneqq \sum_r \sum_{s' \in \mathcal{S}} r\, p(s', r \mid s, a), \tag{6}$$

which on the tile below X is

$$r(s, \uparrow) = 0.8 \cdot 10 + 0.1 \cdot (-10) + 0.1 \cdot 0 = 7. \tag{7}$$

On the terrain the reward is fixed by the tile entered, so $p(s' \mid s, a)$ already contains everything. The joint form (3) is needed when the reward is random even for a known next state, as in a card game where the same hand can win or lose.

3.3 The Markov decision process

A *Markov decision process* (MDP) collects these ingredients into the tuple $(\mathcal{S}, \mathcal{A}, p, \gamma)$, the states, the actions, the dynamics $p(s', r \mid s, a)$, which contain the rewards, and the discount

factor γ of Section 3.4, a number $0 \le \gamma \le 1$ by which a reward is multiplied for every step that it lies in the future. The word decision refers to the agent's actions, which change the probabilities of what comes next. The dynamics always exist, but the agent usually does not know them and only sees samples (s, a, r, s'). If the agent knows p, or learns an approximation of it, it can plan by computing the consequences of actions before taking them, which is called *model-based* reinforcement learning. If it learns from experienced transitions alone, the learning is *model-free*. For the rover the model is known, so exact answers can be computed (Sections 5 and 6) and the model-free methods of the later sections can be checked against them.

3.4 Policy, trajectory and return

The policy is the conditional probability

$$\pi(a \mid s) \coloneqq p(a_t = a \mid s_t = s) \tag{8}$$

of choosing action a in state s. A deterministic policy puts probability one on a single action in every state, and we then write $a = \pi(s)$. For $\pi_\uparrow$ on the tile below X, $\pi_\uparrow(\uparrow \mid s) = 1$. For the random policy $\pi(a \mid s) = 1/4$ for every action. Finding the best policy is the aim of reinforcement learning.

Given the state $s_t = s$, a policy and the dynamics, the continuation of the trajectory up to the random end time T of the episode, $\tau = (a_t, r_{t+1}, s_{t+1}, a_{t+1}, r_{t+2}, \dots, s_T)$, has the *trajectory probability*

$$p_\pi(\tau \mid s_t = s) \coloneqq \prod_{k=t}^{T-1} \pi(a_k \mid s_k)\, p(s_{k+1}, r_{k+1} \mid s_k, a_k), \tag{9}$$

a product of factors chosen by the agent and factors chosen by the environment. A change of the policy changes only the agent's factors, while the dynamics of the environment stay the same. On the terrain the reward is fixed by the tile entered, and each environment factor reduces to the state-transition probability $p(s_{k+1} \mid s_k, a_k)$.

The total reward (1) treats a reward now and a reward many steps later as equal, and it is not finite for tasks that never end. The quantity to be maximised is therefore the *discounted return* from time t,

$$G_t \coloneqq \sum_{k=0}^{T-t-1} \gamma^k\, r_{t+k+1} = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots, \qquad 0 \le \gamma \le 1, \tag{10}$$

and for a trajectory τ we write $G_t(\tau)$. For a task that never ends we set $T = \infty$, which is allowed when $\gamma < 1$. The *discount factor* γ has three roles. For $\gamma < 1$ the sum is finite even for $T = \infty$, provided the rewards are bounded, since $|r_t| \le r_{\max}$ gives $|G_t| \le r_{\max} \sum_{k \ge 0} \gamma^k = r_{\max}/(1-\gamma)$. A reward k steps ahead counts with the weight γ^k, so among strategies that reach the same reward the faster one is preferred. And γ sets the horizon over which the agent plans. Since $\gamma^k \approx e^{-k(1-\gamma)}$ for γ close to 1, the weight falls to $1/e$ after about $1/(1-\gamma)$ steps, 10 steps for $\gamma = 0.9$ and 100 steps for $\gamma = 0.99$. In the limit $\gamma \to 0$ the agent is myopic and takes small early rewards. As $\gamma \to 1$ it accepts low early rewards for a larger reward later. Different values of γ can make different policies optimal, so γ belongs to the definition of the task. In practice it is often chosen together with the learning algorithm, as a horizon long enough for the task. The rover uses $\gamma = 0.9$.

Taking the first term out of the sum in (10) gives the recursion

$$G_t = r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) = r_{t+1} + \gamma\, G_{t+1}. \tag{11}$$

symbol	name	meaning	rover
–	agent	learner and decision maker	the rover
–	environment	everything the agent interacts with	terrain with slips
s_t	state	situation of the environment at step t	tile (row, column)
–	observation	the part of the state the agent receives	the full tile
a_t	action	choice of the agent at step t	one of four directions
r_{t+1}	reward	number returned after a_t	$+10$ at X, -10 at C, else 0
–	terminal state	state that ends the episode	X and C
–	episode	one run from the start to a terminal state	from S to X or C
$s_0, a_0, r_1, s_1, \dots$	trajectory	states, actions and rewards in order	one recorded drive
$R_{\text{tot}} = r_1 + \cdots + r_T$	undiscounted total reward	sum of the rewards of an episode, T random	10 for every episode ending in X
$\mathcal{S}$	state space	all states of the environment	24 tiles
$\mathcal{A}$	action space	all actions of the agent	$\uparrow, \downarrow, \leftarrow, \rightarrow$
(2)	Markov property	next state and reward depend only on s_t, a_t	slips depend only on tile and direction
$p(s', r \mid s, a)$	dynamics, model	where the agent lands and which reward it gets	below X, $\uparrow$: (X, $+10$) 0.8, (C, -10) 0.1, stay 0.1
$p(s' \mid s, a)$	state-transition probability	dynamics with the reward summed out	below X, $\uparrow$: X 0.8, C 0.1, stay 0.1
$r(s, a)$	expected reward	average reward of the pair (s, a)	below X, $\uparrow$: 7
$(\mathcal{S}, \mathcal{A}, p, \gamma)$	Markov decision process	the problem to be solved	the terrain
–	model-based, model-free	learning with or without the use of p	both possible, p is known
$\pi(a \mid s)$	policy	probability of action a in state s	$\pi_\uparrow$, $\pi_\rightarrow$, random
$p_\pi(\tau \mid s_t{=}s)$	trajectory probability	product of agent and environment factors	probability of one path
$G_t = r_{t+1} + \gamma G_{t+1}$	discounted return	discounted sum of future rewards	$0.9^7 \cdot 10 \approx 4.78$ for the shortest path
γ	discount factor	weight γ^k of a reward k steps ahead	0.9

Table 1: The vocabulary of reinforcement learning, in the order of Sections 2 and 3.

The return equals the next reward plus the discounted return from the next step on. At the end of an episode no rewards remain, and we set $G_T = 0$, so that the recursion also holds for $t = T - 1$. For an episode with rewards $0, 0, 0, 10$ and $\gamma = 0.9$, evaluating (11) from the end backwards gives $G_3 = 10$, $G_2 = 9$, $G_1 = 8.1$ and $G_0 = 7.29$. Each earlier time step inherits the final reward, reduced by one factor of γ per step. For the random episode of Figure 2 the last reward reaches the first step as $G_0 = 0.9^{16} \cdot 10 \approx 1.85$, and the eight-step path of a good policy would give $0.9^7 \cdot 10 \approx 4.78$. This backward propagation of a late reward is the mechanism by which reinforcement learning addresses credit assignment, and the recursion (11) is the identity on which the Bellman equations, temporal-difference learning and the policy gradient all rest.

3.5 Summary of the vocabulary

Table 1 collects the vocabulary. The MDP fixes the problem, and a policy is a candidate solution. The next step is to say how good a policy is.

4 Value functions

4.1 Comparing two policies

To improve a policy, and eventually to find the best one, we must first be able to say how good a given policy is. Which of the two policies of Figure 3 is better depends on where the rover stands. A policy whose path passes next to C is risky from tiles near C and harmless from tiles far from it. A single number per policy cannot capture this. We need a number for every state that says how good it is to be there when following a given policy. This number is the value of the state.

4.2 The value of a state as an average return

The idea is most easily seen on a chain of states $s_1 \to s_2 \to s_3 \to \mathrm{X}$, with a reward $+10$ on entering the terminal state X, $\gamma = 0.9$, and a policy π that always moves right. Everything is deterministic, so the value V_π of a state is the return from it, and the recursion (11) gives $V_\pi(s_3) = 10$, $V_\pi(s_2) = 0.9 \cdot 10 = 9$ and $V_\pi(s_1) = 0.9 \cdot 9 = 8.1$. Now let s_2 be slippery. From s_2 the rover reaches s_3 with probability 0.8 and slips into a crater with probability 0.2, which gives -10 and ends the episode. The return from s_2 is now random, $0 + 0.9 \cdot 10 = 9$ with probability 0.8 or -10 with probability 0.2, and the natural measure of how good s_2 is, is the average,

$$V_\pi(s_2) = 0.8 \cdot (0 + 0.9 \cdot 10) + 0.2 \cdot (-10) = 7.2 - 2 = 5.2, \qquad V_\pi(s_1) = 0 + 0.9 \cdot 5.2 = 4.68. \quad (12)$$

The computation of $V_\pi(s_2)$ already has the structure of a recursion for values, an average over what can happen next of the reward plus the discounted value of the next state.

On the terrain the same idea applies with many more possible futures. Under $\pi_\uparrow$ every episode from S follows a different path because of the slips (Figure 4, left). The shortest path to X takes eight steps, so the largest possible return is $0.9^7 \cdot 10 \approx 4.78$. A slip that pushes the rover off a shortest path lengthens the episode and lowers the return, and an episode that slips into C has a negative return. The histogram of 2000 simulated returns (Figure 4, right) has a sharp upper edge at 4.78 and a tail of negative returns. Its mean is the value of S under $\pi_\uparrow$, estimated as $V_{\pi_\uparrow}(S) \approx 2.96 \pm 0.06$, where the uncertainty is the standard deviation of the

2000 returns divided by $\sqrt{2000}$. Estimating a value by averaging simulated returns is called the *Monte Carlo* method.

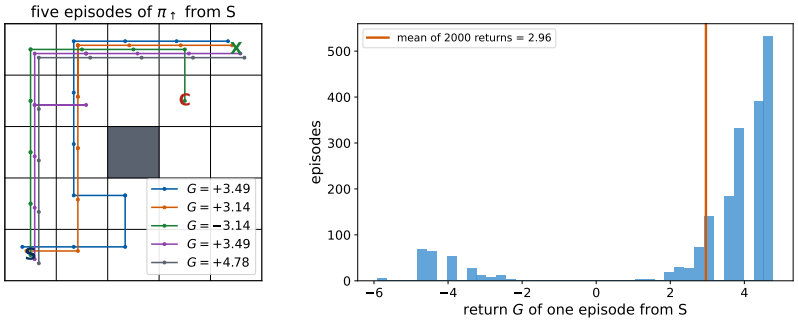


Figure 4: The left panel shows five episodes of $\pi_\uparrow$ from S, each with its return. Slips make the paths differ. The right panel shows a histogram of the returns of 2000 episodes. Its mean is the Monte Carlo estimate of $V_{\pi_\uparrow}(S)$.

4.3 Definitions

The *state-value function* of a policy π is the expected return when the agent starts in s and follows π,

$$V_\pi(s) \coloneqq \mathbb{E}\big[G_t \mid s_t = s, \pi\big] = \sum_\tau p_\pi(\tau \mid s_t = s)\, G_t(\tau), \tag{13}$$

where the sum runs over all continuations τ from $s_t = s$, weighted by their probabilities (9). The value depends on the policy because the future rewards depend on the future actions. It does not depend on t, because neither the policy nor the dynamics do, a property called stationarity. Throughout the chapter V_π and Q_π, with the policy as subscript, denote these true values, and letters without a policy subscript denote other functions of the states. The *action-value function* fixes the first action and follows π afterwards,

$$Q_\pi(s, a) \coloneqq \mathbb{E}\big[G_t \mid s_t = s,\ a_t = a,\ \pi\big]. \tag{14}$$

$V_\pi(s)$ says how good it is to be in s, and $Q_\pi(s, a)$ says how good it is to take a in s. Only Q compares the actions available in a state, which is what an agent without a model needs in order to choose. Choosing with V alone would require predicting where each action leads, which needs $p(s' \mid s, a)$. Their difference, the *advantage*

$$A_\pi(s, a) \coloneqq Q_\pi(s, a) - V_\pi(s), \tag{15}$$

says how much better or worse action a is than what the policy does on average in s.

On the tile below X the policy $\pi_\uparrow$ moves up. Its value there is $V_{\pi_\uparrow}(s) = 7.69$. The action value of $\uparrow$ equals this value, $Q_{\pi_\uparrow}(s, \uparrow) = 7.69$, so the advantage of the action the deterministic policy takes is $A_{\pi_\uparrow}(s, \uparrow) = 0.0$. For a deterministic policy $V_\pi(s) = Q_\pi(s, \pi(s))$ by definition. The action $\leftarrow$ drives into the crater with probability 0.8 and has $Q_{\pi_\uparrow}(s, \leftarrow) = -6.53$, hence the advantage $A_{\pi_\uparrow}(s, \leftarrow) = -14.22$. The Bellman equations of Section 5 yield these numbers, in Sections 5.2 and 5.3. A negative advantage marks an action worse than the policy's choice, and improving a policy means switching to actions with positive advantage.

5 The Bellman equations

5.1 Derivation

Definition (13) is an average over all trajectories. Estimating it by simulation, as in Figure 4, costs 2000 episodes for an uncertainty of ± 0.06 at a single tile, and every tile and every policy needs its own simulation. The Bellman equation replaces this average over whole futures by a relation between the value of a state and the values of its possible successors, one equation per state.

The derivation rests on the Markov property (2) and on *stationarity*, which states that the dynamics p and the policy π do not depend on t, and that episodes end by entering a terminal state and not at a fixed time. Under stationarity the distribution of the future from a state is the same whenever the state is reached, so the value is one function of the state alone. The Bellman equation is a *derived result*, which follows from the definition (13), the recursion (11), the Markov property and stationarity, and from nothing else.

Insert the recursion into the definition and evaluate the expectation in two stages, first over the action of the agent and then over the answer of the environment,

$$
\begin{aligned}
V_\pi(s) &\overset{(a)}{=} \mathbb{E}\big[G_t \mid s_t = s, \pi\big] \overset{(b)}{=} \mathbb{E}\big[r_{t+1} + \gamma G_{t+1} \mid s_t = s, \pi\big] \\
&\overset{(c)}{=} \sum_a \pi(a \mid s)\, \mathbb{E}\big[r_{t+1} + \gamma G_{t+1} \mid s_t = s, a_t = a, \pi\big] \\
&\overset{(d)}{=} \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \Big(r + \gamma\, \mathbb{E}\big[G_{t+1} \mid s_t = s, a_t = a, s_{t+1} = s', r_{t+1} = r, \pi\big]\Big) \\
&\overset{(e)}{=} \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \Big(r + \gamma\, \mathbb{E}\big[G_{t+1} \mid s_{t+1} = s', \pi\big]\Big) \\
&\overset{(f)}{=} \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big(r + \gamma\, V_\pi(s')\big).
\end{aligned}
\tag{16}
$$

Step (a) is the definition. Step (b) uses the recursion (11). Steps (c) and (d) are the law of total expectation, applied twice. In (c) the average is split according to the first action a, which has the probability $\pi(a \mid s)$. In (d) the average for a given action is split according to the next state s' and reward r, which have the probability $p(s', r \mid s, a)$. Given these outcomes, $r_{t+1} = r$ is a fixed number and leaves the inner expectation. Step (e) uses the Markov property to remove s_t, a_t and r_{t+1} from the condition, because the whole future after s_{t+1} depends on the earlier history only through s_{t+1} (Appendix A.1). Step (f) uses stationarity. The expectation in (e) is taken from time $t+1$, and since the future from s' has the same distribution at every time, it equals the value $V_\pi(s')$ of definition (13). A terminal state has no future, and its value is zero, $V_\pi(s_{\text{terminal}}) = 0$, consistent with $G_T = 0$.

Some environments end an episode after a fixed number of steps, the lander of Section 9.2 after 1000 steps. Such a time limit is not a terminal state, so the last state of an episode cut by the limit is not given the value zero. Its value is still estimated from its successors, which treats the limit as external to the task [7].

Equation (16) is the *Bellman expectation equation* [6]. It states that the value of a state equals the average, over the agent's action and the environment's response, of the reward plus the discounted value of the next state, and it generalises the computation of $V_\pi(s_2)$ in (12) to any MDP.

Assumption, definition, derived result

The Markov property (2) and stationarity are assumptions about the environment and the policy. The value function (13) is a definition. The Bellman equation (16) is a theorem that follows from them. If the assumption fails, for example because the agent observes only part of the state, the Bellman equation in terms of the observations does not hold, and the methods built on it lose their guarantees.

5.2 Action values and the relation between V and Q

The same derivation with the first action fixed removes the sum over a,

$$Q_\pi(s,a) = \mathbb{E}\big[r_{t+1} + \gamma G_{t+1} \mid s_t = s, a_t = a, \pi\big] = \sum_{s',r} p(s', r \mid s, a)\big(r + \gamma\, V_\pi(s')\big)$$
$$= r(s,a) + \gamma \sum_{s'} p(s' \mid s, a) V_\pi(s'), \tag{17}$$

where the second line uses (5) and (6). Comparing (17) with (16) gives $V_\pi(s) = \sum_a \pi(a \mid s)\, Q_\pi(s,a)$, so the value of a state is the policy-weighted average of its action values, and the advantage averages to zero under the policy, $\sum_a \pi(a \mid s) A_\pi(s,a) = 0$. On the tile below X, $Q_{\pi_\uparrow}(s,\uparrow) = r(s,\uparrow) + \gamma \cdot 0.1 \cdot V_{\pi_\uparrow}(s) = 7 + 0.09\, V_{\pi_\uparrow}(s)$, because the terminal states X and C have value zero and only the slip to the right leaves the rover on its tile. Since $\pi_\uparrow$ takes this action, $V_{\pi_\uparrow}(s) = Q_{\pi_\uparrow}(s,\uparrow)$, so $V_{\pi_\uparrow}(s) = 7/0.91 = 7.69$.

5.3 Solving the equations with a known model

When the model is known, the Bellman equations for a fixed policy are a linear system. Splitting (16) into a reward part and a value part, as in (17), gives

$$V_\pi(s) = \sum_a \pi(a \mid s)\, r(s,a) + \gamma \sum_{s'} \Big(\sum_a \pi(a \mid s)\, p(s' \mid s, a) \Big) V_\pi(s') = r^\pi_s + \gamma \sum_{s'} P^\pi_{ss'} V_\pi(s'), \tag{18}$$

with the policy-averaged reward vector $r^\pi_s = \sum_a \pi(a \mid s)\, r(s,a)$ and transition matrix $P^\pi_{ss'} = \sum_a \pi(a \mid s)\, p(s' \mid s, a)$. For a terminal state we set the row of P^π and the entry of r^π to zero, because nothing follows it, so its equation reads $V_\pi = 0$. In vector form, with one component per state, the system is $V_\pi = r^\pi + \gamma P^\pi V_\pi$, and with the identity matrix I its solution is

$$V_\pi = (I - \gamma P^\pi)^{-1} r^\pi = \sum_{k=0}^{\infty} \gamma^k (P^\pi)^k r^\pi. \tag{19}$$

The rows of P^π are non-negative and sum to at most one, so for $\gamma < 1$ the matrix γP^π shrinks the largest component of every vector, the series converges and the inverse exists (Appendix A.2). The series shows the meaning of the solution, since $(P^\pi)^k r^\pi$ is the expected reward k steps ahead, and the value is its discounted sum.

For the terrain the system has 24 equations, one per state. For $\pi_\uparrow$ it gives $V_{\pi_\uparrow}(S) = 2.9$, consistent with the Monte Carlo estimate 2.96 ± 0.06 from 2000 simulated episodes. One linear solve replaces all simulations and gives every tile at once, for instance $V_{\pi_\uparrow} = 5.27$ on the tile (2,4) below the tile below X. With it the action value of $\leftarrow$ on the tile below X follows from (17). Moving left enters C with probability 0.8, the slip up enters X with probability 0.1 and the slip down reaches (2,4) with probability 0.1, so $r(s,\leftarrow) = -7$ and $Q_{\pi_\uparrow}(s,\leftarrow) = -7 + 0.9 \cdot 0.1 \cdot 5.27 = -6.53$, the number quoted in Section 4.

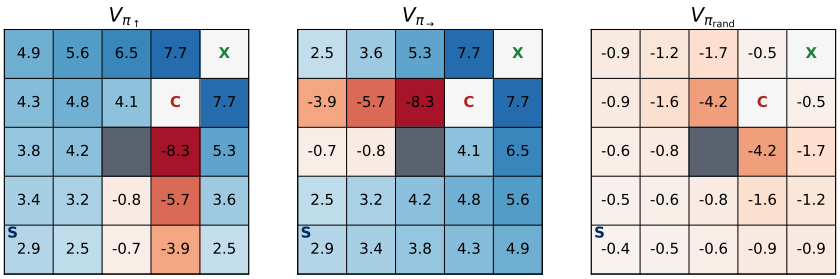


Figure 5: Value maps from the linear system (19) for $\pi_\uparrow$, $\pi_\rightarrow$ and the random policy. Blue tiles have positive value, red tiles negative value. The number in each tile is V_π.

Figure 5 compares the policies. Both hand-made policies have the same value at the start, $V_{\pi_\uparrow}(S) = V_{\pi_\rightarrow}(S) = 2.9$, because reflection on the diagonal through S and X maps the terrain onto itself and exchanges the two policies. Elsewhere they differ. $\pi_\uparrow$ is better on 9 tiles and $\pi_\rightarrow$ on 9. They are equal on the remaining four non-terminal tiles, on (4,0) and (3,1), which lie on the diagonal, and on (0,3) and (1,4), where both policies move towards X. Each is poor on the tiles from which its path runs into the crater, where values turn red. The random policy has a negative value almost everywhere, $V_{\pi_{\text{rand}}}(S) = -0.37$. All rewards arrive at the end of the drive. The value of the start is therefore 10 times the probability of ending at X minus 10 times the probability of ending in C, each weighted by the discount accumulated along the drive. Under the random policy the rover performs a Markov chain on the tiles with the transition matrix P^π, and the chain ends, or is absorbed, in X or in C. The chain started in s ends in C either by entering C in its first step or by moving to a non-terminal tile s' and ending in C from there, so the probability $h(s)$ of ending in C obeys

$$h(s) = P^\pi_{s\text{C}} + \sum_{s'} P^\pi_{ss'}\, h(s'), \tag{20}$$

with the sum over the non-terminal tiles. Equation (20) is a linear system of the same form as (19), with $\gamma = 1$ and the one-step probability of entering C in place of the reward. From S it gives the probability 0.8 of ending in the crater and 0.2 of reaching the sample site, so the penalty outweighs the reward and the value of the start is negative.

Value functions therefore define a *partial ordering* of policies, $\pi \geq \pi'$ if $V_\pi(s) \geq V_{\pi'}(s)$ for every state. The two hand-made policies cannot be ordered, since each is better on some tiles. This leaves open whether a single policy exists that is at least as good as every other in every state, and how to find it.

6 Optimal policies, policy iteration and value iteration

6.1 Optimal policies

A policy π^* is *optimal* if

$$V_{\pi^*}(s) \geq V_\pi(s) \qquad \text{for every state } s \text{ and every policy } \pi. \tag{21}$$

The definition asks for a single policy that is best in all states at once, and the partial ordering does not guarantee that one exists. If two policies are optimal, each is at least as good as the other in every state, so their values coincide. The *optimal value function* V_{π^*} is therefore one function, whichever optimal policy is used to define it, and the same holds for the optimal action values Q_{π^*}. This section derives the equations that V_{π^*} and Q_{π^*} satisfy, shows that

an optimal policy exists, and gives two algorithms that find it. The arguments rest on three assumptions, which also underlie the linear system (19) and hold to the end of Section 7. The MDP is finite, with finitely many states and actions. The rewards are bounded, $|r| \le r_{\max}$. The discount satisfies $\gamma < 1$. Every value is then bounded, $|V_\pi(s)| \le r_{\max}/(1-\gamma)$. Terminal states have value zero for every policy, and all value functions of this section are set to zero on them.

6.2 The Bellman optimality equations

The values of a policy contain the information needed to improve it. Let π be a policy and π' the deterministic policy that is greedy with respect to Q_π, $\pi'(s) = \arg\max_a Q_\pi(s,a)$. The *policy improvement theorem* states that the greedy policy is at least as good in every state,

$$V_{\pi'}(s) \;\ge\; Q_\pi(s,\pi'(s)) = \max_a Q_\pi(s,a) \;\ge\; V_\pi(s) \qquad \text{for every state } s. \tag{22}$$

The right inequality holds because $V_\pi(s) = \sum_a \pi(a \mid s) Q_\pi(s,a)$ is an average of the action values (Section 5), and an average never exceeds the maximum. The left inequality states that following π' throughout is at least as good as taking the greedy action once and following π afterwards. Taking the greedy action once is at least as good as following π, and repeating this comparison at every later step adds only non-negative gains. Appendix A.3 gives the proof.

Applied to an optimal policy π^*, the theorem fixes the optimal values. With $\pi = \pi^*$, optimality gives $V_{\pi'}(s) \le V_{\pi^*}(s)$ for the leftmost member of the chain (22), while its rightmost member is $V_{\pi^*}(s)$. All members of the chain are therefore equal, and in particular

$$V_{\pi^*}(s) = \max_a Q_{\pi^*}(s,a) \qquad \text{for every state } s. \tag{23}$$

Since $V_{\pi^*}(s)$ is also the π^*-weighted average of $Q_{\pi^*}(s,\cdot)$, an optimal policy chooses only actions that maximise $Q_{\pi^*}(s,\cdot)$. The action values of π^* obey (17) like those of any policy. Inserting (17) into (23) gives an equation for V_{π^*} alone, and inserting (23) at the next state into (17) gives an equation for Q_{π^*} alone,

$$V_{\pi^*}(s) = \max_a \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma V_{\pi^*}(s')\big], \tag{24}$$

$$Q_{\pi^*}(s,a) = \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma \max_{a'} Q_{\pi^*}(s',a')\big]. \tag{25}$$

Equations (24) and (25) are the *Bellman optimality equations*. The maximum makes them nonlinear, since which action is maximal depends on the values being sought, so they cannot be solved by a single matrix inversion. The derivation assumed that an optimal policy exists. Policy iteration constructs one, and value iteration computes V_{π^*} directly from (24).

6.3 Policy iteration

Policy iteration alternates two steps. The evaluation step computes the values of the current policy. It is convenient to write the right-hand side of the Bellman equation (16) as an operator $\mathcal{T}_\pi$ that maps a value function V to a new value function,

$$(\mathcal{T}_\pi V)(s) \coloneqq \sum_a \pi(a \mid s) \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma V(s')\big] \tag{26}$$

on the non-terminal states and $(\mathcal{T}_\pi V)(s) = 0$ on the terminal ones. The Bellman equation states $V_\pi = \mathcal{T}_\pi V_\pi$, and the evaluation step solves this linear system by (19) and computes

Q_π from (17). The improvement step replaces the policy by the greedy one, which keeps the current action whenever it is among the maximisers. By (22) the values never decrease, and they increase strictly in every state where the policy changes (Appendix A.3), so no policy occurs twice. A finite MDP has finitely many deterministic policies, $|\mathcal{A}|^{|\mathcal{S}|}$, so policy iteration stops after finitely many improvements.

It stops when the improvement step returns the current policy π. In every state $\pi(s)$ then maximises $Q_\pi(s,\cdot)$, so $V_\pi(s) = Q_\pi(s,\pi(s)) = \max_a Q_\pi(s,a)$, and with (17) this is the optimality equation (24) for V_π. Such a policy is optimal. To compare it with any other policy π', consider

$$(\mathcal{T}_{\pi'} V_\pi)(s) = \sum_a \pi'(a \mid s)\, Q_\pi(s,a), \tag{27}$$

the expected return of acting once with π' in s and following π afterwards. It is an average of the action values and hence at most their maximum,

$$V_\pi(s) \;\geq\; (\mathcal{T}_{\pi'} V_\pi)(s) \qquad \text{for every state } s. \tag{28}$$

A single step with π' therefore never raises the return. Applying the same comparison at every later step shows that switching to π' for any number of steps, and therefore for good, never raises it either (Appendix A.3). Hence $V_\pi(s) \geq V_{\pi'}(s)$ in every state for every policy π', which is the definition (21). An optimal policy therefore exists, and policy iteration finds one. Started from $\pi_\uparrow$ on the terrain, it stops after 3 improvements, and the value of the start rises with every improvement until it reaches the optimum.

6.4 Value iteration

Value iteration solves the optimality equation (24) as a fixed-point equation, without evaluating any policy completely. Written with the expected reward (6) and the state-transition probability (5), its right-hand side defines the *Bellman optimality operator*

$$(\mathcal{T}V)(s) \coloneqq \max_a \Big[r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, V(s') \Big] \tag{29}$$

on the non-terminal states, with $(\mathcal{T}V)(s) = 0$ on the terminal ones, and (24) states $V_{\pi^*} = \mathcal{T}V_{\pi^*}$. Value iteration starts from $V_0(s) = 0$ in every state and repeats the sweep

$$V_{k+1} = \mathcal{T}V_k \tag{30}$$

until the largest change $\max_s |V_{k+1}(s) - V_k(s)|$ falls below a chosen tolerance. The agent then acts greedily, choosing in each state the action that attains the maximum in (29).

On the tile $s = (1,4)$ below X the first sweep gives $V_1(s) = \max_a r(s,a)$, since $V_0 = 0$. The expected rewards of the four actions are 7 for $\uparrow$, 1 for $\rightarrow$ (only the slip up reaches X), -1 for $\downarrow$ (only the slip left reaches C) and -7 for $\leftarrow$, so $V_1(s) = 7$. The tile (2,4) below it has no terminal neighbour, so $V_1(2,4) = 0$. In the second sweep the action $\uparrow$ gives $7 + 0.9 \cdot 0.1 \cdot V_1(s) = 7 + 0.09 \cdot 7 = 7.63$, where the factor 0.1 is the probability of slipping right and staying on the tile. The action $\rightarrow$ stays on the tile with probability 0.8 and reaches (2,4) with probability 0.1, giving $1 + 0.9\,(0.8 \cdot 7 + 0.1 \cdot 0) = 6.04$. Hence $V_2(s) = 7.63$. The third sweep gives $7(1 + 0.09 + 0.09^2) = 7.69$ to two decimals, and the sweeps sum the geometric series $7 \sum_k 0.09^k = 7/0.91$, the optimal value $V_{\pi^*}(1,4) = 7.69$ of this tile. Tiles farther from X receive nonzero values one sweep later per step of distance, so the information about the rewards spreads backwards one step per sweep.

Value iteration converges because every sweep shrinks the error by at least the factor γ. We measure the distance between two value functions by the largest difference over the states,

$\|V - U\| = \max_s |V(s) - U(s)|$. The rewards are the same for V and for U and cancel in the difference $\mathcal{T}V - \mathcal{T}U$. What remains is a difference of next-state values, averaged with the probabilities $p(s' \mid s, a)$ and multiplied by γ, and neither the average nor the maximum over actions enlarges it (Appendix A.4). Hence

$$\|\mathcal{T}V - \mathcal{T}U\| \leq \gamma \, \|V - U\|. \tag{31}$$

An operator with this property is a *contraction*. It has exactly one fixed point, and the iteration converges to it from any start (Appendix A.4). The optimal values V_{π^*} are a fixed point by (24), so they are the only solution of the optimality equation, and value iteration approaches them with the error bound

$$\|V_k - V_{\pi^*}\| \leq \gamma^k \, \|V_0 - V_{\pi^*}\|. \tag{32}$$

The operator $\mathcal{T}_\pi$ of (26) is a contraction in the same sense, with the only fixed point V_π, and so is the right-hand side of (25) on tables of action values, with the only fixed point Q_{π^*}.

Policy iteration and value iteration solve the same problem by two schedules. Policy iteration evaluates each policy completely, one linear solve, before it improves the policy, and it needs few iterations. Value iteration makes only one sweep of evaluation before each improvement, and the maximum in (29) performs the improvement inside the sweep. It needs many cheap sweeps and no matrix inversion. The iteration $V \leftarrow \mathcal{T}_\pi V$ for a fixed policy likewise solves the linear system (19) when the matrix is too large to invert. On the terrain the values of the final policy of policy iteration coincide with those of value iteration to machine precision.

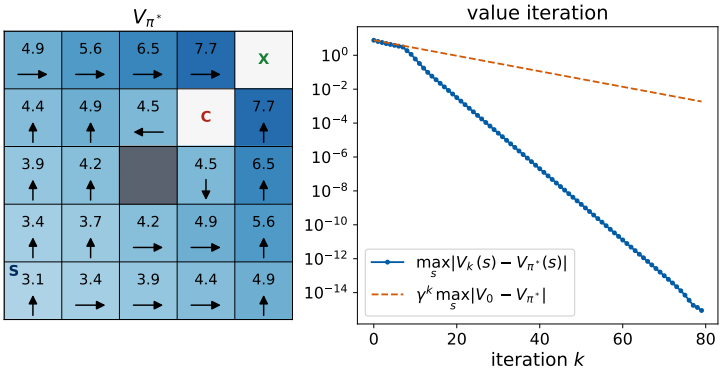


Figure 6: The left panel shows optimal values V_{π^*} and the optimal action on each tile. The right panel shows the largest error of value iteration, $\max_s |V_k(s) - V_{\pi^*}(s)|$, against the iteration number k on a logarithmic scale (blue), and the bound γ^k times the initial error (orange).

Figure 6 shows the result on the terrain. The error stays below the bound (32) and reaches machine precision. The optimal value of the start is $V_{\pi^*}(S) = 3.05$, higher than the 2.9 of both hand-made policies, and V_{π^*} is at least as large as $V_{\pi_\uparrow}$ and $V_{\pi_\rightarrow}$ on every tile. From the tile below C the optimal policy moves down, away from C, since moving up or sideways risks a slip into C. In the second row it moves away from C and reaches X through the top row. The optimal policy accepts a longer path where the short one runs along the crater.

The optimal action values follow from V_{π^*} by (17). On the tile $s = (1, 4)$ below X, with $V_{\pi^*}(1, 4) = 7.69$ and $V_{\pi^*}(2, 4) = 6.53$, they are $Q_{\pi^*}(s, \uparrow) = 7.69$, $Q_{\pi^*}(s, \rightarrow) = 7.13$, $Q_{\pi^*}(s, \downarrow) = 4.39$ and $Q_{\pi^*}(s, \leftarrow) = -6.41$. Any deterministic policy π that chooses in every state an action maximising $Q_{\pi^*}(s, \cdot)$ is optimal, since $(\mathcal{T}_\pi V_{\pi^*})(s) = Q_{\pi^*}(s, \pi(s)) = V_{\pi^*}(s)$ makes V_{π^*} the fixed point of $\mathcal{T}_\pi$, which is V_π. When several actions share the largest Q_{π^*}, any of them may be chosen, so the optimal policy need not be unique. Because Q_{π^*} already contains the

whole optimal future, choosing greedily with respect to it is optimal for the whole future, and the greedy action ↑ is read off the table without any knowledge of the slip probabilities.

All methods so far used the model $p(s', r \mid s, a)$. In most applications the model is unknown, and the remaining sections learn values or policies from sampled experience.

7 Learning values from samples

7.1 A running mean of the Bellman right-hand side

A rover on a real planet, a lander or a chemical reactor does not come with a table $p(s', r \mid s, a)$. The agent experiences transitions (s, a, r, s') one at a time, and we want the solution of the Bellman equation from such samples. Temporal-difference (TD) learning is the bridge from the Bellman equation to learning from samples. It performs the iteration of Section 6 with one sampled transition in place of the average over the next step, and Q-learning and deep Q-learning are built on it. From here on, V and Q without a policy subscript denote the estimates that an algorithm stores and updates.

Written with random variables, the Bellman equation (16) states that the value is a mean,

$$V_\pi(s) = \mathbb{E}\big[r_{t+1} + \gamma V_\pi(s_{t+1}) \mid s_t = s, \pi\big], \tag{33}$$

where the average runs over the action chosen by the policy and the response of the environment. With a model we compute this average. Without a model we can only observe samples of the random number inside it.

Suppose that samples $x_1, x_2, \ldots$ of a random number with mean μ arrive one at a time. A single estimate m suffices to learn μ, if after each sample it is moved a fraction α of the way towards the sample,

$$m \leftarrow m + \alpha\,(x_n - m). \tag{34}$$

With $\alpha = 1/n$ this is the exact sample average, $m_n = \frac{n-1}{n} m_{n-1} + \frac{1}{n} x_n = \frac{1}{n} \sum_{k \le n} x_k$. With a constant α it is a moving average that weights recent samples more and fluctuates around μ, the less the smaller α is (Figure 7). The update never needs the distribution of x. The step sizes of the chapter follow one convention. The symbol α denotes the step size of tabular updates such as (34) and the TD and Q-learning updates built on it, and η the step size of a gradient step on the parameters of a policy or a network.

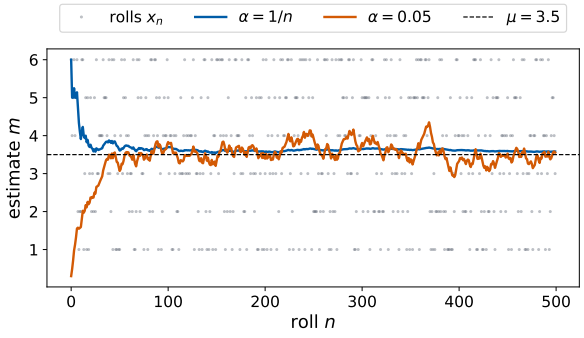


Figure 7: Running mean (34) of simulated die rolls with mean $\mu = 3.5$, with $\alpha = 1/n$, which converges, and with the constant $\alpha = 0.05$, which keeps fluctuating around μ.

One observed transition (s, a, r, s') provides one sample of $r + \gamma V_\pi(s')$, the number inside the average (33), except that $V_\pi(s')$ is unknown, since it is what we want to learn. We replace it by

the current guess $V(s')$ and apply the running mean (34). This gives the temporal-difference update TD(0) [8],

$$V(s) \leftarrow V(s) + \alpha\big[\, r + \gamma V(s') - V(s) \,\big], \qquad \delta = r + \gamma V(s') - V(s). \tag{35}$$

The quantity $r + \gamma V(s')$ is the *TD target* and δ the *TD error*. The TD error compares two predictions of the same return made one step apart, $V(s)$ before the step and $r + \gamma V(s')$ after it, and the later one already contains one real reward. Because the target uses the current estimate, the method is said to *bootstrap*.

As a numerical example, take the optimal policy on the terrain with $\alpha = 0.1$ and all estimates zero. In the first episode every target is zero until the rover, on the tile (0,3) left of X, drives right and enters X. The target is $10 + \gamma V(\mathrm{X}) = 10$, since X is terminal, and the update gives $V(0,3) \leftarrow 0 + 0.1\,(10 + 0 - 0) = 1.0$. After this episode (0,3) is the only tile with a nonzero value. When the rover in the next episode drives from (0,2) to (0,3), the target is $0 + 0.9 \cdot 1.0 = 0.9$, and $V(0,2)$ moves from 0 to 0.09, using the guess at (0,3) in place of the unknown true value.

7.2 Convergence

The target $r + \gamma V(s')$ uses the estimate itself, so it moves while V is learned. Averaged over the randomness of the next step, the target is $(\mathcal{T}_\pi V)(s)$ of (26), the right-hand side of the Bellman equation evaluated with the current estimate. On average TD(0) therefore performs value iteration for the fixed policy, damped by the step size, and since $\mathcal{T}_\pi$ is a contraction this iteration approaches V_π. With step sizes that decrease in a suitable way and every state visited again and again, the sampled updates converge to V_π with probability one [10], under the conditions stated in Appendix A.5. With a constant α the estimate fluctuates around the solution, as the moving average of the die does.

Bootstrapping is not circular

Every step injects one real reward, and the contraction shrinks the errors that the target borrows from the estimate.

7.3 The value spreads backwards

Figure 8 shows TD(0) with $\alpha = 0.1$ evaluating the optimal policy on the terrain from $V = 0$. After one episode only the tile (0,3) carries a value, the 1.0 of the first update. Each later visit carries the value one tile further back along the path, which is the recursion (11) applied by learning. Tiles off the usual path, reached only through slips, fill in later. After 1000 episodes the map approaches the exact values. The remaining differences come from the fluctuations left by a constant step size and from tiles that the optimal policy rarely visits.

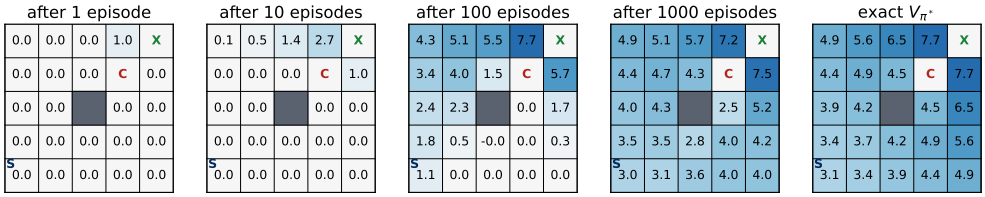


Figure 8: TD(0) evaluation of the optimal policy on the terrain with $\alpha = 0.1$ after 1, 10, 100 and 1000 episodes, starting from $V = 0$, and the exact values V_{π^*} (last panel).

7.4 Monte Carlo and temporal difference

The Monte Carlo method of Section 4 is also a running mean, with the full return as target, $V(s) \leftarrow V(s) + \alpha\,[G_t - V(s)]$. Its target is an unbiased sample of the value and does not need the Markov property, but it is available only at the end of the episode and collects the randomness of every later step. The TD target is available after every step and contains the randomness of one step only, at the price of a bias while the estimates are still wrong. Figure 9 compares the two methods on the random policy, for two step sizes and ten random seeds each. For both step sizes TD reaches a smaller error than Monte Carlo after the same number of episodes. The larger step size leaves both methods at a larger error, the fluctuation of a moving average with a constant step. This ranking is observed for this problem and these step sizes, and the general reason for it is the lower variance of the TD target.

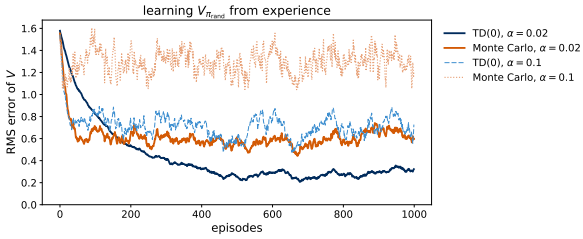


Figure 9: Root-mean-square error of TD(0) and Monte Carlo estimates of the values of the random policy against the exact values, as a function of the number of episodes, mean over ten seeds. Dark solid lines $\alpha = 0.02$, light dashed (TD) and dotted (Monte Carlo) lines $\alpha = 0.1$.

7.5 Q-learning

TD evaluates a given policy. To find the optimal policy without a model, apply the same idea to the optimality equations. Both can be written as means,

$$V_{\pi^*}(s) = \max_a \mathbb{E}\big[r + \gamma V_{\pi^*}(s') \mid s, a\big], \qquad Q_{\pi^*}(s,a) = \mathbb{E}\Big[r + \gamma \max_{a'} Q_{\pi^*}(s', a') \;\Big|\; s, a\Big]. \tag{36}$$

For V_{π^*} the maximum stands outside the expectation. One sampled transition of one action cannot estimate a maximum over all actions, and even with V_{π^*} known, choosing an action would require $p(s' \mid s, a)$. For Q_{π^*} the expectation stands outside and runs only over the environment's response to the pair (s, a), while the maximum over a' is computed by the agent from its own table. One transition (s, a, r, s') is therefore a sample of the right-hand side, with the current table in place of the unknown Q_{π^*}, and the running mean applies. The resulting update is *Q-learning* [13],

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[\, r + \gamma \max_{a'} Q(s', a') - Q(s,a) \,\big], \tag{37}$$

with the target r when s' is terminal. Averaged over the response of the environment, the target is the optimality operator for action values applied to the current table, a γ-contraction with fixed point Q_{π^*} (Section 6.4), so the convergence argument of Section 7.2 applies. Q-learning converges to Q_{π^*} with probability one when every state–action pair is visited infinitely often and the step sizes of each pair satisfy the conditions of Appendix A.5 [13].

As a numerical example, start with a table of zeros and $\alpha = 0.1$. The first time the rover on (0,3) chooses $\rightarrow$ and enters X, the update gives $Q((0,3), \rightarrow) \leftarrow 0 + 0.1\,(10 - 0) = 1.0$. When it later drives right from (0,2) to (0,3), the target is $0 + 0.9 \max_{a'} Q((0,3), a') = 0.9 \cdot 1.0$, and

$Q((0,2),\rightarrow)$ becomes 0.09. The target uses the largest entry of the row of (0,3), whichever action the rover takes there next.

Q-learning thus runs in a loop. The agent observes the state s, chooses an action a by the ε-greedy rule of the next subsection, takes it and observes r and s', updates $Q(s,a)$ by (37), and repeats from s' until the episode ends.

7.6 Acting while learning

The agent must choose actions while it learns, and it faces the dilemma of *exploration and exploitation* [1]. To collect reward it should exploit, choosing the action with the largest current Q. To discover whether other actions are better, and to keep every pair updated, it must explore. Pure exploration is a random search that never uses what it learned, and pure exploitation commits to the first strategy that works, however poor. The simplest compromise is the *ε-greedy* policy, which takes a random action with probability ε and the action with the largest Q otherwise. In practice ε often starts large and is lowered during training.

Q-learning is *off-policy*. The agent behaves ε-greedily, but the target (37) uses the maximum, the value of acting greedily from s', so it learns the values of the optimal policy while it acts partly at random. The on-policy variant SARSA, named after the quintuple (s,a,r,s',a') that it uses, updates $Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma Q(s',a') - Q(s,a)]$ with the action a' actually taken next, and it learns the value of the ε-greedy behaviour itself.

7.7 Q-learning on the terrain

After 5000 episodes with $\varepsilon = 0.1$ and $\alpha = 0.1$, the greedy action of the learned table agrees with the optimal policy on 20 of the 22 non-terminal tiles (Figure 10). The tiles where it differs do not lie on the optimal path from S, which climbs the first column, and on them the chosen action is only slightly worse than the optimal one. The greedy policy $\hat{\pi}$ of the table, evaluated with the known model, therefore comes close to the optimum at the start, $V_{\hat{\pi}}(S) = 3.03$ against $V_{\pi^*}(S) = 3.05$. The return per episode during training stays mostly below $V_{\pi^*}(S)$, because ten percent of the actions are random and occasionally lead into the crater. Tiles far from the usual path are visited rarely and keep less accurate values.

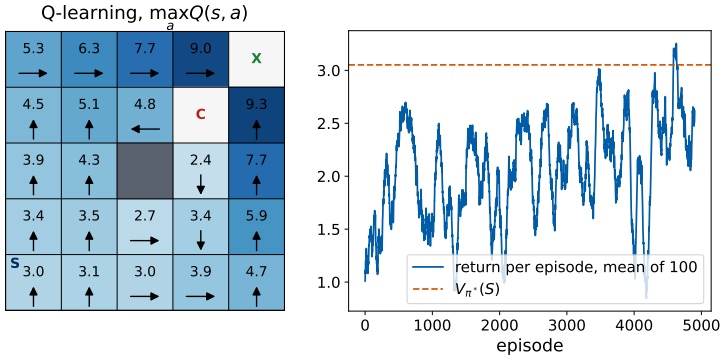


Figure 10: Q-learning on the terrain after 5000 episodes. The left panel shows the learned $\max_a Q(s,a)$ with the greedy action on each tile. The greedy action differs from the optimal one of Figure 6 on the tiles (3,3), where it points down, and (4,1), where it points up. The right panel shows the return per episode during training and the optimal value $V_{\pi^*}(S)$.

In this run some learned values near X exceed the optimal values of Figure 6. A known mechanism contributes to this excess. With a constant step size every estimate fluctuates

around its target, and the maximum over four fluctuating estimates tends to pick those that are high by chance,

$$\mathbb{E}\big[\max_a Q(s,a)\big] \geq \max_a \mathbb{E}\big[Q(s,a)\big], \tag{38}$$

so the target is too large on average and the excess propagates through the targets of the neighbouring tiles. This effect is called the *overestimation bias* of Q-learning.

7.8 Exploration on a second grid

The need for exploration is most evident on a deterministic grid of 3×7 tiles without slips and with $\gamma = 0.95$ (Figure 11). The agent starts on the bottom row, two steps right of a small reward $+1$ in the bottom-left corner, and six steps from a large reward $+10$ in the top-right corner. Both reward tiles end the episode, so an episode collects one of the two. From the start the near reward is worth $0.95 \cdot 1 = 0.95$ and the far one $0.95^5 \cdot 10 \approx 7.7$, so the optimal policy walks to the $+10$.

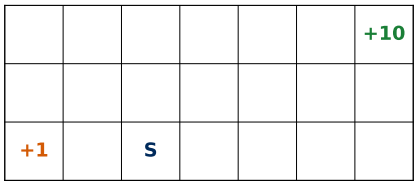


Figure 11: The 3×7 exploration grid. The agent starts at S, the small reward $+1$ lies two steps to the left, the large reward $+10$ six steps away in the top-right corner. Moves are deterministic.

We ran Q-learning for 2000 episodes with 20 random seeds for each ε and counted the runs whose final greedy policy walks to the $+10$ (Figure 12, right). The greedy agent with $\varepsilon = 0$ finds the far reward in 0 of the 20 runs, and the count grows with ε up to 20 of 20 for $\varepsilon = 0.5$. The greedy agent moves at random only while all its values are equal. Once it has found the $+1$, the actions along that path have positive values and all other actions still have value zero, so the greedy choice repeats the path in every later episode, and the far reward is never discovered. With $\varepsilon > 0$ the agent keeps taking random actions, and eventually a chain of them reaches the $+10$, the sooner the larger ε. Exploration also has a cost, the return lost to random actions after the good path is known. At the end of training the discounted return per episode stays below the return 7.74 of the optimal path for every ε (Figure 12, left), because random moves lengthen the path and sometimes end the episode at the $+1$.

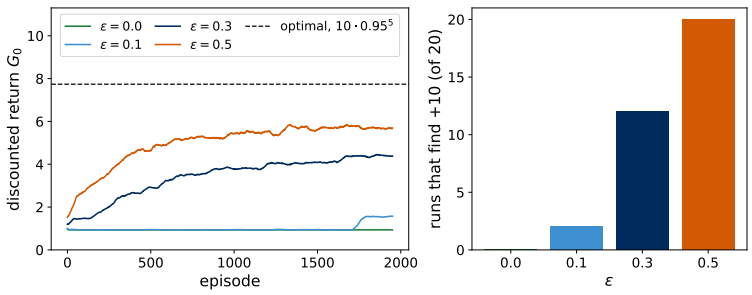


Figure 12: Exploration on the 3×7 grid. The left panel shows the discounted return G_0 per episode during training for four values of ε, mean over 20 seeds and running mean over 50 episodes. The dashed line marks the return 7.74 of the optimal path. The right panel shows the number of runs out of 20 whose final greedy policy walks to the $+10$.

8 Policy gradients

8.1 Learning the policy directly

Q-learning derives its policy from values, $a = \arg\max_a Q(s,a)$. Many problems in science have continuous actions, such as a coil voltage, a laser amplitude or a joint torque, for which the maximisation over actions would be a separate optimisation at every step. The optimal behaviour can also be stochastic, for instance when the agent observes only part of the state, and an arg max yields only deterministic policies. And a small change of the values can swap the largest action and change the behaviour abruptly. Policy-gradient methods avoid these limitations by parametrising the policy itself, a function $\pi_\theta(a \mid s)$ with parameters θ that outputs the probabilities of the actions, and by optimising the expected return directly,

$$J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\big[G_0(\tau)\big] = \sum_\tau p_\theta(\tau)\, G_0(\tau), \qquad p_\theta(\tau) \coloneqq p(s_0)\, p_{\pi_\theta}(\tau \mid s_0), \tag{39}$$

where τ is a whole episode, $p(s_0)$ the probability of the start state and $p_{\pi_\theta}(\tau \mid s_0)$ the trajectory probability (9) with $t = 0$ under π_θ. The parameters can be a table of numbers, one per state and action, or the weights of a neural network (Section 9). For continuous actions the policy outputs the parameters of a distribution, for example the mean and width of a Gaussian, and the formulas of this section apply with the density of the action in place of its probability.

8.2 The policy gradient

The parameters enter J only through the trajectory probabilities. The *log-derivative trick* $\nabla_\theta p_\theta = p_\theta\, \nabla_\theta \ln p_\theta$ converts the gradient of an average into an average,

$$\nabla_\theta J = \sum_\tau \nabla_\theta p_\theta(\tau)\, G_0(\tau) = \sum_\tau p_\theta(\tau)\, \nabla_\theta \ln p_\theta(\tau)\, G_0(\tau) = \mathbb{E}_{\tau\sim p_\theta}\big[\nabla_\theta \ln p_\theta(\tau)\, G_0(\tau)\big], \tag{40}$$

which can be estimated by playing episodes and averaging. The logarithm of $p_\theta(\tau)$ is a sum,

$$\ln p_\theta(\tau) = \ln p(s_0) + \sum_t \ln \pi_\theta(a_t \mid s_t) + \sum_t \ln p(s_{t+1}, r_{t+1} \mid s_t, a_t), \tag{41}$$

and only the policy terms depend on θ, so $\nabla_\theta \ln p_\theta(\tau) = \sum_t \nabla_\theta \ln \pi_\theta(a_t \mid s_t)$. The unknown dynamics drop out, and the physics of the environment is never differentiated and never needs to be known, as the separation of agent and environment factors in (9) anticipated. The vector $\nabla_\theta \ln \pi_\theta(a \mid s)$ is called the *score* of the action.

Averaged over the action, the score vanishes in every state,

$$\mathbb{E}_{a\sim\pi_\theta}\big[\nabla_\theta \ln \pi_\theta(a \mid s)\big] = \sum_a \pi_\theta(a \mid s) \frac{\nabla_\theta \pi_\theta(a \mid s)}{\pi_\theta(a \mid s)} = \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0. \tag{42}$$

Inserting $\nabla_\theta \ln p_\theta(\tau) = \sum_t \nabla_\theta \ln \pi_\theta(a_t \mid s_t)$ and $G_0 = \sum_k \gamma^k r_{k+1}$ into (40) gives

$$\nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \ln \pi_\theta(a_t \mid s_t) \sum_k \gamma^k r_{k+1}\Big], \tag{43}$$

in which the score of every action multiplies every reward of the episode. An action at time t receives no credit for rewards that arrived before it, because such rewards are fixed when a_t is drawn and multiply a score of mean zero (Appendix A.6). Dropping these terms leaves for each t the rewards from time t on, which add up to the discounted return from t, reduced by t factors of γ,

$$\sum_{k\ge t} \gamma^k r_{k+1} = \gamma^t \sum_{j\ge 0} \gamma^j r_{t+j+1} = \gamma^t G_t, \tag{44}$$

with $k = t + j$ and the definition (10) of G_t. Hence

$$\nabla_\theta J = \mathbb{E}\Big[\sum_t \gamma^t \nabla_\theta \ln \pi_\theta(a_t \mid s_t)\, G_t\Big] = \mathbb{E}\Big[\sum_t \gamma^t \nabla_\theta \ln \pi_\theta(a_t \mid s_t)\, Q_{\pi_\theta}(s_t, a_t)\Big]. \tag{45}$$

Equation (45) is the *policy gradient*, the exact gradient of the objective (39), and its second form is the *policy gradient theorem* [17]. The second form follows because averaging G_t over everything that happens after (s_t, a_t) gives $Q_{\pi_\theta}(s_t, a_t)$ by definition (14) (Appendix A.6). In practice the factor γ^t is dropped, which gives the direction

$$\hat{g} \coloneqq \mathbb{E}\Big[\sum_t \nabla_\theta \ln \pi_\theta(a_t \mid s_t)\; G_t\Big], \tag{46}$$

in which each action is weighted by the *reward-to-go* G_t, the return from its own time on. REINFORCE and all experiments of this chapter use $\hat{g}$, and for $\gamma < 1$ it is not exactly the gradient of J (Appendix A.6 and Ref. [14]).

8.3 REINFORCE

The simplest algorithm built on (46) is REINFORCE [15]. It repeats three steps. Play N episodes with the current policy π_θ. Compute the reward-to-go G_t of every step from the recursion (11), backwards from the end of each episode. Update

$$\theta \leftarrow \theta + \eta \,\frac{1}{N} \sum_{\text{episodes}} \sum_t \nabla_\theta \ln \pi_\theta(a_t \mid s_t)\, G_t \tag{47}$$

with a learning rate η. In a software library the same step is a gradient descent on the loss

$$L(\theta) = -\frac{1}{N} \sum_{\text{episodes}} \sum_t \ln \pi_\theta(a_t \mid s_t)\, G_t, \tag{48}$$

in which G_t is treated as a constant and the double sum over all steps of the batch is divided by the number of episodes N. It resembles the cross-entropy of a classifier whose labels are the actions taken, each weighted by the return that followed. Actions followed by a large return become more probable. The estimate (47) is noisy, because the return of a whole episode collects the randomness of every later step and every later action.

8.4 A two-action example

The smallest problem in which every quantity of the policy gradient can be computed exactly has one state and two actions a_1 and a_2 (Figure 13). Each episode consists of a single action, which ends it with a deterministic reward, $R(a_1) = 2$ for the better action and $R(a_2) = 1$ for the worse one. The return of an episode is therefore $G_0 = R(a)$. The policy has one parameter θ and chooses a_1 with the probability

$$\pi_\theta(a_1) = \sigma(\theta) =: p, \qquad \pi_\theta(a_2) = 1 - p, \qquad \sigma(\theta) = \frac{1}{1 + e^{-\theta}}, \tag{49}$$

the logistic function, which is a softmax of the two preferences θ and 0. The objective (39) and its exact gradient follow from $\mathrm{d}\sigma/\mathrm{d}\theta = \sigma(1 - \sigma)$,

$$J(\theta) = 2p + (1 - p), \qquad \frac{\mathrm{d}J}{\mathrm{d}\theta} = p(1 - p). \tag{50}$$

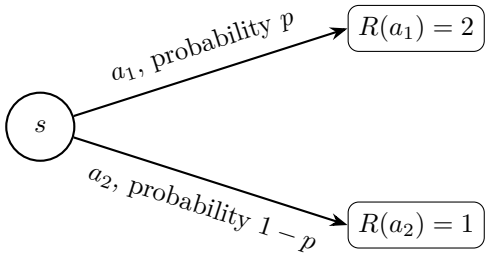


Figure 13: The two-action problem. From the single state s the policy chooses a_1 with probability $p = \sigma(\theta)$ and a_2 with probability $1 - p$. Each action ends the episode with the reward shown.

The gradient is positive for every θ, so raising θ, and with it the probability of the better action, always increases the expected return.

REINFORCE estimates this gradient from one sampled action. The scores of the two actions are

$$\frac{\partial}{\partial\theta}\ln\pi_\theta(a_1) = 1 - p, \qquad \frac{\partial}{\partial\theta}\ln\pi_\theta(a_2) = -p, \tag{51}$$

and the single-sample estimate $g = R(a)\,\partial_\theta \ln\pi_\theta(a)$ takes the value $2(1-p)$ with probability p and $-p$ with probability $1 - p$. Its mean, $2p(1-p) - p(1-p) = p(1-p)$, is the exact gradient (50), as the policy gradient (45) guarantees. At $\theta = 0$, where $p = 1/2$ and the exact gradient is 0.25, the estimate is $g = 1$ when a_1 is drawn and $g = -0.5$ when a_2 is drawn. A step $\theta \leftarrow \theta + \eta g$ therefore raises θ after a_1 and lowers it after a_2, so it raises the probability of whichever action was sampled. The sign of the step is set by the score, and since both rewards are positive the reward only sets its size. A positive return never lowers the probability of the action that produced it. The policy still improves on average, because the better action receives the larger push, 1 against 0.5, and the average step is the exact gradient 0.25. Each single estimate, however, misses the mean by 0.75 in one direction or the other, and its variance is 0.5625.

8.5 The baseline

The useful information lies in whether an action was followed by more or less return than usual. Subtracting from G_t the typical return from the same state, a *baseline* $b(s_t)$, centres the weights. Actions followed by more return than usual are made more probable, actions followed by less are made less probable, and actions with ordinary outcomes change the policy little. A baseline that depends on the state but not on the action leaves the mean of the gradient unchanged, because by (42)

$$\mathbb{E}\big[\nabla_\theta \ln\pi_\theta(a_t \mid s_t)\, b(s_t)\big] = \mathbb{E}_{s_t}\Big[b(s_t)\sum_a \pi_\theta(a \mid s_t)\nabla_\theta \ln\pi_\theta(a \mid s_t)\Big] = 0, \tag{52}$$

while its variance changes. Averaging G_t over everything that happens after (s_t, a_t) gives $Q_{\pi_\theta}(s_t, a_t)$ by definition (14), so the weight $G_t - b(s_t)$ has the conditional mean $Q_{\pi_\theta}(s_t, a_t) - b(s_t)$ given s_t and a_t.

The two-action problem shows both properties exactly. With a constant baseline b the single-sample estimate becomes $g_b = (R(a) - b)\,\partial_\theta \ln\pi_\theta(a)$, with mean $(2-b)(1-p)p - (1-b)p(1-p) = p(1-p)$ for every b. At $p = 1/2$ it takes the two values $(2-b)/2$ and $(b-1)/2$ with equal probability, so its variance is the square of half their difference,

$$\mathrm{Var}[g_b] = \frac{(3-2b)^2}{16} \qquad \text{at } \theta = 0. \tag{53}$$

Table 2 lists three choices. Without a baseline the variance is 0.5625. The baseline $b = 1.5$, the mean reward $J(0)$ under the current policy, makes both actions give $g = 0.25$, every single sample equals the exact gradient, and the variance vanishes. The sampled action a_1, better than average, is pushed up, and a_2, worse than average, is pushed down. The baseline $b = 10$ lies far above both rewards, so every sampled action is pushed down and the estimate jumps between -4 and 4.5. Its mean is still 0.25, while its variance has grown to about 18. A baseline can therefore reduce the variance, and the reduction is largest near the average return, whereas a badly chosen baseline increases it.

baseline b	g_b when a_1 is drawn	g_b when a_2 is drawn	mean of g_b	variance of g_b
0	1	-0.5	0.25	0.5625
1.5	0.25	0.25	0.25	0
10	-4	4.5	0.25	18.0625

Table 2: Single-sample REINFORCE estimates $g_b = (R(a) - b)\,\partial_\theta \ln \pi_\theta(a)$ for the two-action problem at $\theta = 0$, where both actions have probability $1/2$ and the exact gradient is 0.25.

In a problem with many states the typical return depends on the state, and the standard baseline is the value function of Section 4, $b(s_t) = V_{\pi_\theta}(s_t)$. The conditional mean of the weight then becomes $Q_{\pi_\theta}(s_t, a_t) - V_{\pi_\theta}(s_t)$, the advantage $A_{\pi_\theta}(s_t, a_t)$ of (15), which says how much better the action was than what the policy does on average in that state. The value V_{π_θ} is unknown and must itself be learned, which Section 9.4 does with a second network. A simpler practical form centres the returns of a batch and divides them by their standard deviation [1]. The batch mean acts as a constant baseline, and the division rescales the step size.

8.6 Softmax policies

A problem with many states needs a policy that assigns probabilities to the actions of every state through a common set of parameters. For discrete actions it is usually a softmax of *action preferences* $x_\theta(s, a)$ [1],

$$\pi_\theta(a \mid s) = \frac{e^{x_\theta(s,a)}}{\sum_b e^{x_\theta(s,b)}}, \qquad \nabla_\theta \ln \pi_\theta(a \mid s) = \nabla_\theta x_\theta(s, a) - \sum_b \pi_\theta(b \mid s)\, \nabla_\theta x_\theta(s, b). \tag{54}$$

The two-action policy (49) is the case of one state with the preferences θ and 0. The score follows from $\ln \pi_\theta(a \mid s) = x_\theta(s, a) - \ln \sum_b e^{x_\theta(s,b)}$, since the gradient of the second term is $\sum_b e^{x_\theta(s,b)} \nabla_\theta x_\theta(s, b) / \sum_c e^{x_\theta(s,c)} = \sum_b \pi_\theta(b \mid s) \nabla_\theta x_\theta(s, b)$. For linear preferences $x_\theta(s, a) = \boldsymbol{\theta}_a \cdot \mathbf{s}$, with one weight vector $\boldsymbol{\theta}_a$ per action and a state vector $\mathbf{s}$, the score with respect to the weights $\boldsymbol{\theta}_b$ of action b is

$$\nabla_{\boldsymbol{\theta}_b} \ln \pi_\theta(a \mid s) = (\mathbf{1}[a = b] - \pi_\theta(b \mid s))\, \mathbf{s}, \tag{55}$$

where the indicator $\mathbf{1}[a = b]$ is 1 for $b = a$ and 0 otherwise. On the terrain the state vector of a tile is one-hot, a vector with one component per tile that equals 1 for the tile on which the rover stands and 0 for all others. The product $\boldsymbol{\theta}_a \cdot \mathbf{s}$ then picks one component of $\boldsymbol{\theta}_a$, so the preferences form a table with one entry per tile and action, and a gradient step changes only the entries of the current tile. When the weight vectors are stacked as the rows of one matrix, the score is the matrix $(\mathbf{e}_a - \mathbf{p})\, \mathbf{s}^\mathsf{T}$, with the unit vector $\mathbf{e}_a$ of the chosen action and the vector $\mathbf{p}$ whose components are the action probabilities $\pi_\theta(b \mid s)$. For two actions a_1, a_2 with $\pi_\theta(a_1 \mid s) = 0.3$ and the action a_1 chosen, the score moves $\boldsymbol{\theta}_{a_1}$ by $0.7\,\mathbf{s}$ and $\boldsymbol{\theta}_{a_2}$ by $-0.7\,\mathbf{s}$, which raises the probability of a_1 in this state. An action that was improbable receives a

larger push than one that was already likely. With a neural network the preferences are its outputs, and the score is computed by backpropagation.

9 Deep networks in place of tables

9.1 When a table is impossible

All methods so far store one number per state, or per state and action, a value $V(s)$, an action value $Q(s, a)$ or an action preference. The terrain has 22 non-terminal states with four actions each, and a table of 88 action values is enough. A table fails in two situations. When the state is continuous, such as the position and velocity of a spacecraft, a state practically never repeats, so almost every state met is new and nothing learned in other states is used. When the state is discrete but the number of states is astronomically large, as the about 10^{170} positions of Go, most states are never visited. In both cases similar states should have similar values, which a table cannot express, since each entry is learned independently of all others.

A neural network replaces the table (Figure 14). The action-value network $Q_\theta(s, a)$ takes the state as input and returns one value for each action, so the greedy action is the index of the largest output. The policy network $\pi_\theta(a \mid s)$ takes the state and returns the action probabilities through the softmax (54), with the network outputs as action preferences. Both are trained with the update rules of the earlier sections, now applied to the weights θ by gradient steps. A step computed for one state also changes the predictions for similar states, and through this generalisation a network learns a large problem from a feasible number of samples. The convergence guarantees of the tabular methods are lost, because a step that corrects one state can spoil another.

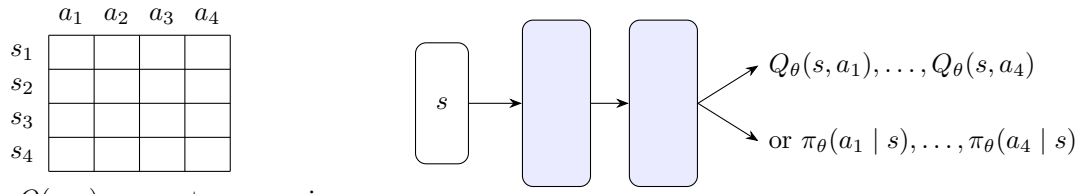


table $Q(s, a)$, one entry per pair

network with weights θ, one input s

Figure 14: A table stores a separate number for every state and action (left). A network computes all action values, or all action probabilities, of a state from the state vector s through hidden layers with shared weights θ (right), so that similar states receive similar outputs.

9.2 Deep Q-learning on the Lunar Lander

The Lunar Lander is a two-dimensional physics simulation (the environment `LunarLander-v3` of the Gymnasium library, built on the Box2D physics engine [16]). A small spacecraft starts near the top of the screen with a random push and must land on a pad between two flags. Its state consists of eight numbers, the position (x, y), velocity (v_x, v_y), angle φ, angular velocity ω, and two indicators of whether each leg touches the ground. The four actions are to do nothing, fire the left engine, fire the main engine, or fire the right engine. An episode ends when the lander crashes, leaves the screen, comes to rest, or after 1000 steps.

The reward of this environment is *shaped*. Besides the final outcome it rewards progress along the way. Moving closer to the pad and slowing down earns reward, tilting costs some, each

leg that touches the ground gives +10, every frame of engine firing costs a little fuel, and the episode ends with +100 for coming to rest or −100 for a crash. A return of at least 200 is the conventional threshold for a successful landing. Shaping addresses credit assignment directly, but it requires care, because a badly designed reward is exploited through behaviour the designer did not intend. An agent that chooses actions at random has a mean return of −203.3 over 50 episodes.

The eight state variables are real numbers, so the table of Q-learning is replaced by a network $Q_\theta(s, \cdot)$ with two hidden layers of 256 units and the ReLU activation, $8 \to 256 \to 256 \to 4$, and 69124 parameters. The tabular update (37) moves $Q(s, a)$ towards the target $r + \gamma \max_{a'} Q(s', a')$. With a network the same step becomes a regression of $Q_\theta(s, a)$ on this target over a mini-batch B of stored transitions (s, a, r, s', d). The target is computed with a second set of parameters θ^-, those of the target network described in the next paragraph, and the regression minimises the squared TD error

$$L(\theta) = \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} \Big(r + \gamma(1 - d) \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a) \Big)^2 . \tag{56}$$

The flag d equals 1 when s' is terminal, after a crash, which includes leaving the screen, or after coming to rest, and the target is then r. At the time limit of 1000 steps $d = 0$, so the target still bootstraps there, which treats the limit as external to the task [7]. The target is treated as a fixed label, and the gradient is taken only through $Q_\theta(s, a)$. For a single transition one gradient step then reads $\theta \leftarrow \theta + \eta\, \delta\, \nabla_\theta Q_\theta(s, a)$, where the TD error δ is the bracket of (56) and a constant factor is absorbed into η. The move towards the target of the tabular update is now distributed over all weights.

The label of this regression depends on the network being trained. With $\theta^- = \theta$ every gradient step changes the targets of all states, so the regression pursues a moving target. Consecutive transitions of an episode are almost identical, so successive steps also push the network repeatedly in the same direction in one region of the state space. This feedback can make the training diverge, because errors in rarely sampled regions are not corrected and bootstrapping feeds them back into the targets elsewhere [17]. Two changes make deep Q-learning (the deep Q-network, DQN [18]) stable in practice. A *replay buffer* stores the most recent transitions and supplies random mini-batches drawn from them, which reduces the correlation between the samples of one gradient step. A *target network* Q_{θ^-}, a copy of the trained network that follows it slowly, computes the targets, so that the label stays nearly fixed over many gradient steps. In the original experiments on Atari games, removing either of the two lowered the scores [18]. All settings of the experiments, together with standard refinements of the loss, are listed in Appendix B.

In every step of training the agent acts ε-greedily with respect to Q_θ, stores the transition (s, a, r, s', d) in the buffer, samples a random mini-batch, takes one gradient step on the loss (56) and moves θ^- a little towards θ.

9.3 The trained lander

The agent was trained for 200 000 environment steps, which took 11.5 minutes on a laptop CPU. The return of the training episodes rises from the level of the random agent to above the landing threshold of 200 (Figure 15). Evaluated without exploration on 50 new episodes, the trained agent has a mean return of 242.3, and 40 of the 50 episodes reach 200. Saved snapshots of the network, run on one fixed test episode, show how the behaviour develops (Table 3). The untrained network rotates uncontrollably and crashes, the network after 25 000 steps hovers above the pad, which avoids the crash penalty but consumes fuel, and the trained network lands under control.

environment steps	0	25 000	50 000	100 000	200 000
return on the test episode	−458.2	−42.4	128.6	211.5	258.2

Table 3: Return of saved snapshots of the action-value network on one fixed test episode of the Lunar Lander.

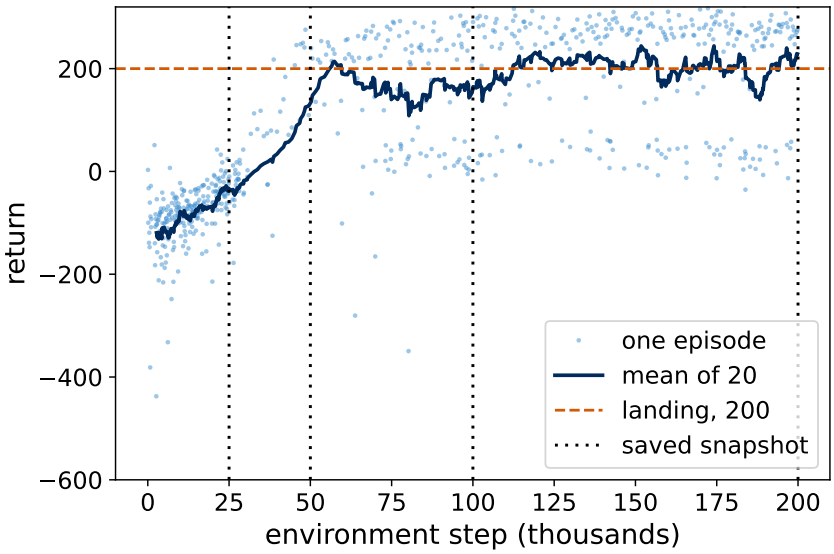


Figure 15: Deep Q-learning on the Lunar Lander. Return of every training episode against the environment step at which it ended (dots), its running mean over 20 episodes (solid line) and the landing threshold 200 (dashed). Dotted vertical lines mark the saved snapshots of Table 3.

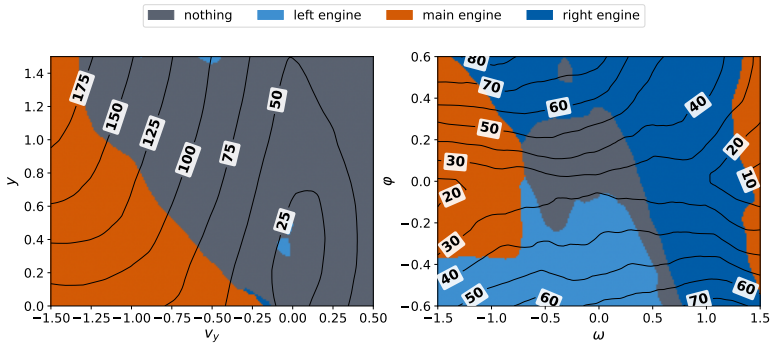


Figure 16: Greedy action of the trained network on two slices through the eight-dimensional state space, with the other variables fixed. Contour lines show the predicted value. The left panel shows the upright lander above the pad, with the height y against the vertical velocity v_y, and the right panel the lander at mid height, with the angle φ against the angular velocity ω.

Figure 16 shows the learned control rule on two slices of the state space. Above the pad the main engine fires when the lander falls fast for its height. At mid height the side engine is chosen by a combination of angle and angular velocity, as for a damped controller, a rule that emerged from the rewards alone.

9.4 A policy network on the cart-pole

The policy gradient is demonstrated on the cart-pole [20], also a simulation (`CartPole-v1` in Gymnasium). A pole is hinged on a cart that moves on a track, and every 0.02 s the agent pushes the cart left or right. The state is four numbers, the cart position x, its velocity $\dot{x}$, the pole angle φ and its angular velocity $\dot{\varphi}$. The reward is $+1$ for every step with the pole up, and the episode ends when the pole tilts beyond 12°, the cart leaves the track ($|x| > 2.4$), or after 500 steps. The undiscounted return is therefore the length of the episode, at most 500, and serves as the measure of performance, while the weights G_t of the gradient estimator are discounted with $\gamma = 0.99$. Random pushes keep the pole up for 23.0 steps on average. The task has two discrete actions, so deep Q-learning would also solve it, and it serves as the simplest test bed for the policy gradient, cheap to simulate and quick to learn.

The policy network $\pi_\theta(a \mid s)$ has two hidden layers of 64 units, $4 \to 64 \to 64 \to 2$, with 4610 parameters, and its two outputs are the action preferences of the softmax (54). It is trained by REINFORCE (47) with one gradient step after every batch of five episodes. Every weight G_t of this task is positive, so, as in the two-action problem, every action taken is pushed up and only the sizes of the pushes differ. The baseline $V_{\pi_\theta}(s_t)$ of Section 8.5 is learned by a second network $V_w(s)$ with parameters w, of the same shape with one output, $4 \to 64 \to 64 \to 1$, and 4545 parameters. After every batch it is fitted by regression on the observed reward-to-go G_t, the Monte Carlo method of Section 4 with a network in place of the table. The weight $G_t - V_w(s_t)$ is computed with the value network as it was before this fit and is treated as a constant when the policy is differentiated. A network already fitted to the returns of the batch would partly reproduce the sampled G_t itself, and the baseline would then depend on the action that produced this return and change the mean of the gradient.

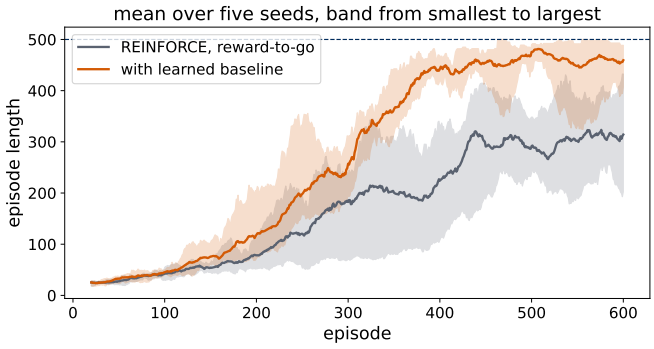


Figure 17: REINFORCE on the cart-pole. Episode length during training without a baseline (reward-to-go G_t as weight) and with the learned baseline ($G_t - V_w(s_t)$ as weight). Lines are the mean over five seeds, bands span the smallest to the largest seed.

Figure 17 compares 600 training episodes with and without the baseline, five random seeds each. With the baseline the episodes become long sooner and the seeds differ less, as expected from the smaller variance of centred weights in the two-action problem (Table 2). Over the last 50 episodes the mean length is 461.1 with the baseline, close to the maximum of 500, and 313.3 without it.

The two networks of this experiment are the starting point of *actor-critic* methods, in which

the policy network is called the actor and the value network the critic. These methods replace the weight $G_t - V_w(s_t)$, which is known only at the end of an episode and carries the noise of every later step, by the TD error $r_{t+1} + \gamma V_w(s_{t+1}) - V_w(s_t)$ of a single step, whose mean is the advantage when the critic is exact. The policy can then be updated after every step, at the price of a bias whenever the critic is wrong [17]. Many deep reinforcement-learning algorithms used in practice belong to this family, including the one of the atom planner in Section 10.4.

9.5 Summary of the methods

Table 4 collects the methods of Sections 5 to 9. The first three need the model and compute exact answers, and the others learn from sampled experience.

method	needs p	what is learned	target or update
linear system	yes	V_π	$V_\pi = (I - \gamma P^\pi)^{-1} r^\pi$, (19)
policy iteration	yes	π^*, V_{π^*}	evaluate V_π, then $\pi(s) \leftarrow \arg\max_a Q_\pi(s,a)$
value iteration	yes	V_{π^*}	$V \leftarrow \mathcal{T}V$, (30)
Monte Carlo	no	V_π	target G_t
TD(0)	no	V_π	target $r + \gamma V(s')$, (35)
Q-learning	no	Q_{π^*}	target $r + \gamma \max_{a'} Q(s',a')$, (37)
SARSA	no	Q of the ε-greedy policy	target $r + \gamma Q(s',a')$
REINFORCE	no	π_θ, optionally a baseline	$\theta \leftarrow \theta + \eta \sum_t \nabla_\theta \ln \pi_\theta(a_t \mid s_t)\,[G_t - b(s_t)]$, (47)
deep Q-learning	no	$Q_\theta \approx Q_{\pi^*}$	loss (56) with replay buffer and target network
REINFORCE with networks	no	π_θ and V_w	weight $G_t - V_w(s_t)$, regression of V_w on G_t

Table 4: The methods of the chapter, whether they need the dynamics $p(s', r \mid s, a)$, what they learn or compute, and their target or update.

10 Applications

10.1 Games

When an exact simulator is available, learned values and policies can be combined with planning, and the best-known successes of reinforcement learning are of this kind, in games where the rules are the simulator and the result of a game is the reward. The board of Go has 19×19 points and about 10^{170} legal positions, and the value of a move often becomes clear only hundreds of moves later. AlphaGo [21] combined a policy network and a value network with Monte Carlo tree search, a look-ahead that expands the most promising moves. Its policy network was first trained by supervised learning on expert games and then improved by playing against itself. It beat Lee Sedol 4:1 in 2016. AlphaGo Zero [22] dropped the expert games and learned from self-play with only the rules. Playing strength is measured by the Elo rating, a scale on which only differences matter and a player rated ΔR points higher is expected to score $1/(1 + 10^{-\Delta R/400})$ points per game. In Figure 18 the network trained by supervised learning on expert games starts with a large lead and levels off near the experts' level, while the network trained by self-play overtakes it and passes AlphaGo Lee, the version

that played Lee Sedol. AlphaZero [23] applied the same algorithm to Go, chess and shogi. Its network has a value head, which predicts the result of the game from a position like a critic, and a policy head, which guides the tree search.

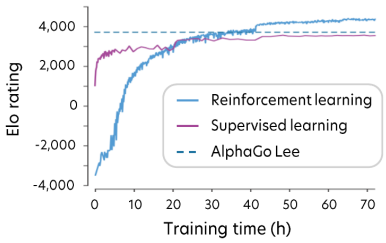


Figure 18: Elo rating against training time for AlphaGo Zero, trained by reinforcement learning from self-play only, and for the same network trained by supervised learning on expert games. The dashed line marks AlphaGo Lee. Figure from [1], after [22].

MuZero [24] plays without being given the rules. It learns a model that predicts, for a move, the next hidden description of the situation and the reward, runs the tree search inside this learned model, and matches AlphaZero in Go, chess and shogi while also playing Atari video games. MuZero is therefore model-based reinforcement learning with a learned model.

10.2 Discovering algorithms

AlphaTensor [3] applies the same machinery to mathematics. Multiplying two matrices is described by a three-index tensor, and every way of writing this tensor as a sum of rank-one terms gives an algorithm whose number of multiplications equals the number of terms. The search becomes a single-player game. The state is the part of the tensor that remains to be represented, an action subtracts one rank-one term, and every step costs a reward of -1, so the return counts the multiplications of the algorithm found. For 4×4 matrices in arithmetic modulo 2 the agent found an algorithm with 47 multiplications, against 49 from applying the construction of Strassen [25] recursively. Because the rules of the game are exact, every algorithm found is a correct algorithm, and the reward measures its cost.

10.3 Treatment strategies

The AI Clinician [4] poses the treatment of sepsis in intensive care as an MDP. The state is built from vital signs and laboratory values of a patient, an action is one of 25 combinations of doses of intravenous fluids and vasopressors, and the reward is given at the end, positive for survival and negative for death. The policy was learned from the records of about 17 000 intensive-care admissions, without any new experiments, and validated on about 79 000 patients of an independent database. In the validation data, patients whose actual doses were close to those of the learned policy had the lowest mortality. The result is debated, because a policy learned only from logged decisions cannot test actions that the physicians rarely took, and its value for such actions must be extrapolated from the records.

10.4 Control in the laboratory

The vocabulary of Section 3 applies wherever an apparatus is adjusted in a sequence of settings and the outcome is measured. The tokamak controller of [2] receives magnetic measurements of the plasma as its state, sets the voltages of the control coils as its action, and was trained only in a simulator before it ran on the device. Quantum devices are a further case [5].

The smallest control problem of a quantum device can be solved exactly, like the rover. A qubit must be brought from $|0\rangle$ to $|1\rangle$. Every pure state of a qubit is a point on the Bloch sphere, with $|0\rangle$ at the north pole and $|1\rangle$ at the south pole, and a microwave pulse rotates this point about an axis. The agent may apply four pulses, rotations by $\pm 90°$ about the x or the y axis. Starting from $|0\rangle$ such rotations reach only six points, the two poles and the four equator states $|\pm\rangle = (|0\rangle \pm |1\rangle)/\sqrt{2}$ and $|\pm i\rangle = (|0\rangle \pm i|1\rangle)/\sqrt{2}$, which form the state space. A pulse works with probability 0.9 and fails with probability 0.1, leaving the state unchanged. Reaching $|1\rangle$ gives the reward $+1$ and ends the episode, every other step gives 0, and $\gamma = 0.9$. From $|0\rangle$ each pulse leads to an equator state. From an equator state one pulse rotates it to $|1\rangle$, one back to $|0\rangle$, and the two pulses about the axis on which the state lies leave it in place.

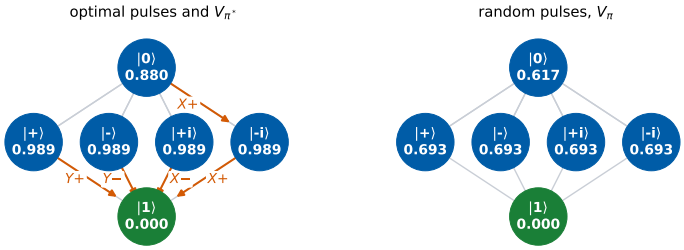


Figure 19: The qubit control problem. Nodes are the six reachable states with their values for $\gamma = 0.9$, grey lines connect states that one successful pulse turns into each other. The left panel shows the optimal policy, with orange arrows marking the chosen pulse ($X\pm$, $Y\pm$ denote $\pm 90°$ rotations about x or y). From $|0\rangle$ all four pulses are equally good and one of them is drawn. The right panel shows random pulses.

By symmetry the Bellman equation (16) for the policy that always applies the pulse towards $|1\rangle$ has two unknowns, the value v_e of the equator states and v_0 of $|0\rangle$,

$$v_e = 0.9 \cdot 1 + 0.1\,\gamma v_e \qquad \Rightarrow \qquad v_e = \frac{0.9}{1 - 0.1\gamma} = 0.989, \tag{57}$$

$$v_0 = 0.9\,\gamma v_e + 0.1\,\gamma v_0 \qquad \Rightarrow \qquad v_0 = \frac{0.9\,\gamma v_e}{1 - 0.1\gamma} = 0.88. \tag{58}$$

On an equator state the pulse about its own axis has the action value $\gamma v_e = 0.890$ and the pulse back to $|0\rangle$ has $0.9\,\gamma v_0 + 0.1\,\gamma v_e = 0.802$, both below v_e. The policy is greedy with respect to its own action values and therefore optimal by Section 6.3. It is the 180° rotation made of two 90° pulses about the same axis, the "π pulse" of the laboratory (a different π from the policy). Random pulses have the lower values 0.617 and 0.693 (Figure 19).

A processor with many qubits needs planning on a much larger scale. In a neutral-atom processor single atoms held by focused laser beams store the qubits, and before every layer of two-qubit gates the atoms must be moved in pairs from a storage zone into a gate zone, each move costing time and adding errors (Figure 20). In Ref. [26] this task is posed as a game in which an action places one atom and the reward is the reduction of an estimated error cost of moves and gates. Two transformer networks, one reading the coming gates and one the planned moves, with separate policy and value heads, are trained by a policy-gradient method on an emulator of the device. In these simulations the trained agent generates a plan for a new circuit without further training, through a sequence of decisions, one atom at a time and one chunk of the circuit after another, and the plan lowers the estimated cost.

In a quantum device the agent usually does not see the state, because a measurement disturbs it and returns only a noisy outcome. In the example of [27] a cavity, a resonator that stores photons, is excited by a linear drive whose amplitude the agent chooses among discretised

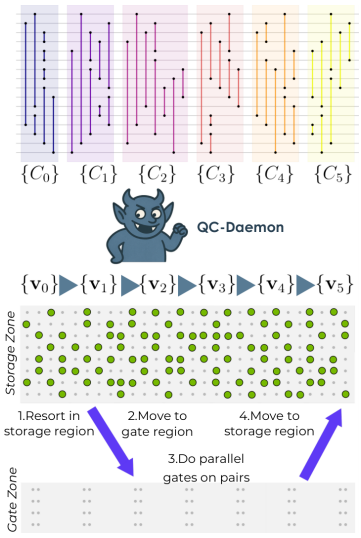


Figure 20: Planning atom moves in a neutral-atom quantum computer, from [26]. The top panel shows a circuit cut into chunks $C_0, \ldots, C_5$ of two-qubit gates (vertical lines connect the two qubits of a gate). In the middle panel the agent proposes a layout $\mathbf{v}_k$ of the atoms in the storage zone for each chunk. In the bottom panel, for each chunk, the atoms are resorted in storage, moved to the gate zone, the gates act on the pairs in parallel, and the atoms return to storage.

values. The agent observes the record of a weak quantum non-demolition measurement of the photon number, a measurement that does not change the photon number while it gradually reveals it. The aim is to prepare the state with exactly one photon and keep it there. The softmax policy and the policy gradient of Section 8 apply unchanged, and during 400 training epochs the probability of one photon grows to about one half (Figure 21c). A linear cavity driven without feedback ends in a coherent state with Poisson-distributed photon number of mean $\bar{n}$, whose one-photon probability $\bar{n}e^{-\bar{n}}$ is at most $e^{-1} \approx 0.37$, and the agent exceeds this coherent-state value by reacting to the measured signal. An agent of this kind was later run in hardware [28], as a neural network on a programmable chip with a latency below a microsecond, which learned from measurements alone to reset a superconducting qubit.

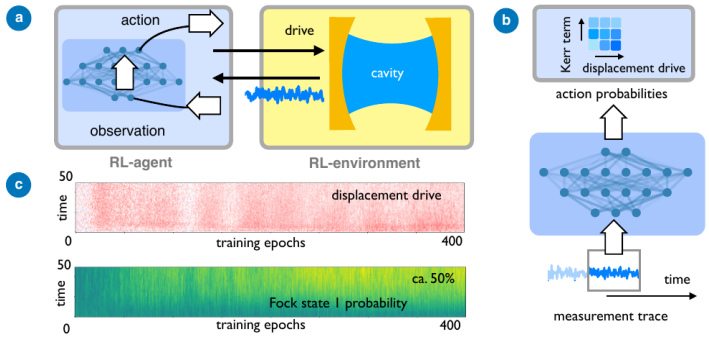


Figure 21: Feedback control of a cavity, from [27]. (a) Scheme of the feedback loop, in which the agent receives a measurement signal from the cavity and sets the control. (b) The network maps the recent measurement trace to probabilities of discretised actions. (c) For a weak photon-number measurement and a linear drive, during 400 training epochs, the drive amplitude (top) and the probability of the one-photon state (bottom) as functions of time within an episode.

In all these examples the algorithms are those of this chapter, with few changes. The scientific work lies in the choice of a state, of actions and of a reward that is informative and inexpensive to evaluate, and in a network adapted to the structure of the state.

A Proofs

A.1 The Markov step of the Bellman derivation

Step (e) of (16) removes s_t, a_t and r_{t+1} from the condition. The return G_{t+1} is built from the continuation $a_{t+1}, s_{t+2}, r_{t+2}, a_{t+2}, \dots$, and the probability of this continuation given the whole past is the product $\pi(a_{t+1} \mid s_{t+1})\, p(s_{t+2}, r_{t+2} \mid s_{t+1}, a_{t+1})\, \pi(a_{t+2} \mid s_{t+2}) \cdots$. The factors of the environment have this form by (2), and those of the agent because the policy depends only on the current state. The first factor depends on the past only through s_{t+1}, and each further factor only on the state generated just before it. By induction over the length of the continuation, its probability, and with it the distribution of G_{t+1}, depends on s_t, a_t and r_{t+1} only through s_{t+1}, so $\mathbb{E}[G_{t+1} \mid s_t, a_t, s_{t+1}, r_{t+1}, \pi] = \mathbb{E}[G_{t+1} \mid s_{t+1}, \pi]$.

A.2 Convergence of the series for V_π

The entries of P^π are non-negative, and each row sums to one for a non-terminal state and to zero for a terminal one. For an eigenvector x of P^π with eigenvalue λ and the component s where $|x_s|$ is largest,

$$|\lambda|\,|x_s| = \Big|\sum_{s'} P^\pi_{ss'} x_{s'}\Big| \le \sum_{s'} P^\pi_{ss'} |x_{s'}| \le |x_s| \sum_{s'} P^\pi_{ss'} \le |x_s|, \tag{59}$$

so the spectral radius of P^π is at most 1 and that of γP^π at most $\gamma < 1$. Hence $I - \gamma P^\pi$ has no zero eigenvalue, and the geometric series in (19) converges. The same estimate in the maximum norm, $\|\gamma P^\pi x\| \le \gamma \|x\|$, gives the convergence of the series directly, since its terms are bounded by $\gamma^k \|r^\pi\|$.

A.3 The policy improvement theorem

Let π be any policy and π' greedy with respect to Q_π. A maximum is at least as large as any average, so $Q_\pi(s, \pi'(s)) \ge \sum_a \pi(a \mid s) Q_\pi(s, a) = V_\pi(s)$ for every s. Writing Q_π with (17) as a conditional expectation and applying the same inequality at the next state gives the chain

$$\begin{aligned}
V_\pi(s) \le Q_\pi(s, \pi'(s)) &\overset{(i)}{=} \mathbb{E}\big[r_{t+1} + \gamma V_\pi(s_{t+1}) \mid s_t = s, a_t = \pi'(s)\big] \\
&\overset{(ii)}{\le} \mathbb{E}\big[r_{t+1} + \gamma Q_\pi(s_{t+1}, \pi'(s_{t+1})) \mid s_t = s, a_t = \pi'(s)\big] \\
&\overset{(iii)}{=} \mathbb{E}\Big[r_{t+1} + \gamma\, \mathbb{E}\big[r_{t+2} + \gamma V_\pi(s_{t+2}) \mid s_{t+1}, a_{t+1} = \pi'(s_{t+1})\big] \;\Big|\; s_t = s, a_t = \pi'(s)\Big] \\
&\overset{(iv)}{=} \mathbb{E}_{\pi'}\big[r_{t+1} + \gamma r_{t+2} + \gamma^2 V_\pi(s_{t+2}) \mid s_t = s\big] \\
&\overset{(v)}{\le} \mathbb{E}_{\pi'}\Big[\sum_{j=0}^{k-1} \gamma^j r_{t+j+1} + \gamma^k V_\pi(s_{t+k}) \;\Big|\; s_t = s\Big] \xrightarrow{k\to\infty} V_{\pi'}(s).
\end{aligned} \tag{60}$$

Here $\mathbb{E}_{\pi'}$ denotes the expectation when all actions are chosen by π'. Step (i) is (17), an average over the next state and reward. Step (ii) uses $V_\pi(s') \le Q_\pi(s', \pi'(s'))$ in every state s', and an average with non-negative weights preserves the inequality. Step (iii) inserts (17) for $Q_\pi(s_{t+1}, \pi'(s_{t+1}))$ one step later, which stationarity allows. The inner expectation runs over s_{t+2} and r_{t+2} for given s_{t+1}. Step (iv) is the tower property of conditional expectations, $\mathbb{E}[\mathbb{E}[Y \mid s_{t+1}] \mid s_t] = \mathbb{E}[Y \mid s_t]$ for a random number Y. By the Markov property the inner expectation, conditioned on s_{t+1} alone, equals the one conditioned on the whole history up to s_{t+1}, so the two nested averages combine into one average over two steps, in both of which

π' chooses the action. Repeating steps (ii) to (iv) k times gives (v). The bracket in (v) differs from the return G_t by $\gamma^k V_\pi(s_{t+k}) - \sum_{j \geq k} \gamma^j r_{t+j+1}$, and each of the two terms is at most $\gamma^k r_{\max}/(1-\gamma)$ in magnitude. As $k \to \infty$ the difference vanishes for $\gamma < 1$, and the bracket becomes $\mathbb{E}_{\pi'}[G_t \mid s_t = s] = V_{\pi'}(s)$. This proves (22).

The chain also shows $V_{\pi'}(s) \geq Q_\pi(s, \pi'(s))$, the left inequality of (22). For a deterministic π the greedy step of policy iteration keeps the current action whenever it is among the maximisers. If $\pi'(s) \neq \pi(s)$ in some state, the old action is therefore not a maximiser, so $Q_\pi(s, \pi'(s)) > V_\pi(s)$, and the chain gives $V_{\pi'}(s) \geq Q_\pi(s, \pi'(s)) > V_\pi(s)$. The improvement is strict wherever the policy changes.

The same reasoning shows that the final policy π of policy iteration is optimal. The operator $\mathcal{T}_{\pi'}$ of (26) averages with non-negative weights and therefore preserves inequalities between value functions, $V \geq U$ implies $\mathcal{T}_{\pi'} V \geq \mathcal{T}_{\pi'} U$. Applying it to both sides of (28) gives $\mathcal{T}_{\pi'} V_\pi \geq \mathcal{T}_{\pi'}^2 V_\pi$, and together with (28) $V_\pi \geq \mathcal{T}_{\pi'}^2 V_\pi$. Repeating this k times gives $V_\pi \geq \mathcal{T}_{\pi'}^k V_\pi$. The function $\mathcal{T}_{\pi'}^k V_\pi$ is the expected return of following π' for k steps and π afterwards. It equals the series (19) for π' cut after k terms plus the term $\gamma^k (P^{\pi'})^k V_\pi$, which vanishes as $k \to \infty$, so $\mathcal{T}_{\pi'}^k V_\pi \to V_{\pi'}$. Hence $V_\pi \geq V_{\pi'}$ in every state for every policy π'.

A.4 The fixed point of value iteration

The operator $\mathcal{T}$ of (29) can be written with the dynamics or with the derived quantities,

$$(\mathcal{T}V)(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\,[r + \gamma V(s')] = \max_a \Big[r(s,a) + \gamma \sum_{s'} p(s' \mid s, a)\, V(s') \Big], \tag{61}$$

since the reward part is $\sum_{s',r} r\, p(s', r \mid s, a) = r(s, a)$ by (6), and in the value part $V(s')$ does not depend on r, so the sum over r gives $p(s' \mid s, a)$ by (5). With the average over actions in place of the maximum it becomes $\mathcal{T}_\pi$ of (26). In this appendix V^* denotes the fixed point of $\mathcal{T}$, whose existence and uniqueness are proved here, and Section 6.4 identifies it with V_{π^*}.

Contraction. For two functions f and g of the action,

$$\big|\max_a f(a) - \max_a g(a)\big| \leq \max_a |f(a) - g(a)|, \tag{62}$$

because with a_1 a maximiser of f we have $\max_a f - \max_a g \leq f(a_1) - g(a_1)$, and the same holds with f and g exchanged. Take for $f(a)$ and $g(a)$ the brackets of (29) for V and for U in a non-terminal state s. The rewards cancel in $f(a) - g(a)$, and the remainder $\gamma \sum_{s'} p(s' \mid s, a)(V(s') - U(s'))$ is at most $\gamma \|V - U\|$ in magnitude, because the probabilities are non-negative and sum to one. With (62) this gives $|(\mathcal{T}V)(s) - (\mathcal{T}U)(s)| \leq \gamma \|V - U\|$ in every non-terminal state, while on the terminal states both sides vanish, which proves (31). With $\sum_a \pi(a \mid s)$ in place of $\max_a$ the same steps show that $\mathcal{T}_\pi$ is a γ-contraction. The right-hand side of (25) is a γ-contraction on tables of action values in the norm $\max_{s,a} |Q(s,a) - Q'(s,a)|$, since (62) applies to $\max_{a'}$ at every next state.

Fixed point. A contraction has at most one fixed point, because $V = \mathcal{T}V$ and $U = \mathcal{T}U$ give $\|V - U\| \leq \gamma \|V - U\|$, which for $\gamma < 1$ forces $V = U$. It also has one. The iteration $V_{k+1} = \mathcal{T}V_k$ from any V_0 satisfies $\|V_{k+1} - V_k\| \leq \gamma^k \|V_1 - V_0\|$, so the V_k form a Cauchy sequence in a finite-dimensional space and converge to a limit V^*, which satisfies $V^* = \mathcal{T}V^*$ because $\mathcal{T}$ is continuous. The argument is the Banach fixed-point theorem for the present case. The error obeys $\|V_k - V^*\| = \|\mathcal{T}V_{k-1} - \mathcal{T}V^*\| \leq \gamma \|V_{k-1} - V^*\|$, and by induction (32). By the same argument the unique fixed point of $\mathcal{T}_\pi$ is V_π.

Optimality. The final policy π of policy iteration puts all probability on a maximiser of its own action values, so $V_\pi = \mathcal{T}_\pi V_\pi = \mathcal{T}V_\pi$ and $V_\pi = V^*$. Section 6 shows that $V_\pi \geq V_{\pi'}$ for every policy π', so the fixed point of $\mathcal{T}$ is the optimal value function V_{π^*}.

A.5 Convergence of temporal-difference learning

Averaged over the action of the policy and the response of the environment, the TD target $r + \gamma V(s')$ in state s is $(\mathcal{T}_\pi V)(s)$, so the averaged TD(0) update applied to all states at once reads

$$V \leftarrow V + \alpha(\mathcal{T}_\pi V - V) = (1-\alpha)V + \alpha\,\mathcal{T}_\pi V. \tag{63}$$

Since $\mathcal{T}_\pi$ is a γ-contraction (Appendix A.4), the triangle inequality gives for $0 < \alpha \le 1$ and two value functions V, U

$$\big\|(1-\alpha)(V-U) + \alpha(\mathcal{T}_\pi V - \mathcal{T}_\pi U)\big\| \le (1-\alpha(1-\gamma))\,\|V-U\|, \tag{64}$$

so the damped map (63) is a contraction with factor $1-\alpha(1-\gamma) < 1$. It converges, and its only fixed point solves $V = \mathcal{T}_\pi V$, the Bellman equation.

The real update differs from (63) in two ways. It changes only the state just visited, and it uses one sample, which adds a fluctuation of mean zero to the averaged step. Let $\alpha_n(s)$ be the step size used at the n-th visit of state s. If every state is visited infinitely often and in every state

$$\sum_n \alpha_n(s) = \infty, \qquad \sum_n \alpha_n(s)^2 < \infty, \tag{65}$$

the estimate converges to V_π with probability one. The first condition ensures that learning never stops too early, and the second that the accumulated noise stays finite. These are the conditions of Robbins and Monro for stochastic approximation [11]. The convergence of TD was proved in Ref. [12] and, with probability one, in Refs. [9, 10]. For Q-learning the same conditions apply to every state–action pair [13].

A.6 Past rewards in the policy gradient

Equation (43) contains the products $r_{k+1}\nabla_\theta \ln \pi_\theta(a_t \mid s_t)$ for all pairs k, t. For $k < t$ the reward r_{k+1} is a function of the history $\mathcal{H}_t = (s_0, a_0, r_1, s_1, \ldots, r_t, s_t)$ only. The expectation over trajectories can be taken first over a_t given $\mathcal{H}_t$ and then over $\mathcal{H}_t$, and given $\mathcal{H}_t$ the action a_t is drawn from $\pi_\theta(\cdot \mid s_t)$, so

$$\mathbb{E}\big[r_{k+1}\nabla_\theta \ln \pi_\theta(a_t \mid s_t)\big] = \mathbb{E}_{\mathcal{H}_t}\Big[r_{k+1}\,\mathbb{E}_{a_t}\big[\nabla_\theta \ln \pi_\theta(a_t \mid s_t) \mid \mathcal{H}_t\big]\Big] = 0 \tag{66}$$

by (42). The remaining terms with $k \ge t$ sum to $\gamma^t G_t$ by (44), which gives the first form of (45). For the second form, condition each term on $\mathcal{H}_t$ and a_t. The score is then fixed, and the remaining average of G_t is $\mathbb{E}[G_t \mid \mathcal{H}_t, a_t] = \mathbb{E}[G_t \mid s_t, a_t] = Q_{\pi_\theta}(s_t, a_t)$, where the first equality is the Markov property (2) of Section 3.1, in the form proved in Appendix A.1, and the second the definition (14). The same conditioning shows that a baseline b may depend on the history $\mathcal{H}_t$, but not on a_t, without changing the mean.

The direction $\hat{g}$ of (46) differs from the gradient (45) by the missing factor γ^t. The objective J counts a decision at time t with the weight γ^t, while $\hat{g}$ weights every visited state as if the episode started there. With $\gamma = 0.99$ and episodes of hundreds of steps the exact weights make late decisions almost irrelevant, which is why most implementations use $\hat{g}$. For $\gamma < 1$ the direction $\hat{g}$ is in general not the gradient of J and need not be the gradient of any function of θ [14]. For $\gamma = 1$ the two expressions coincide.

B Numerical methods

All networks are implemented in PyTorch [29] and trained with the Adam optimiser [30] on a laptop CPU, and the environments are those of the Gymnasium library [16].

For deep Q-learning on the Lunar Lander the replay buffer holds the last 100 000 transitions, each mini-batch contains 64 transitions, and gradient steps start after the first 1000 transitions. The exploration rate ε is lowered linearly from 1 to 0.05 over the first 50 000 of the 200 000 environment steps. The learning rate is $5 \cdot 10^{-4}$ and $\gamma = 0.99$. The target network follows the trained network by the soft update $\theta^- \leftarrow (1-\nu)\theta^- + \nu\theta$ with $\nu = 0.005$ after every step, on a time scale of $1/\nu = 200$ steps. The original DQN copied θ into θ^- every few thousand steps [18]. The loss (56) is modified in two ways. Double DQN [19], which counters the overestimation bias of Section 7.7, replaces the maximum in the target by $Q_{\theta^-}(s', a^*)$ with $a^* = \arg\max_{a'} Q_\theta(s', a')$. The square of the TD error δ is replaced by the Huber function, $\frac{1}{2}\delta^2$ for $|\delta| \le 1$ and $|\delta| - \frac{1}{2}$ beyond, whose derivative with respect to δ is bounded by 1 in magnitude. The gradient with respect to the weights can still be large, since the network multiplies this derivative by $\nabla_\theta Q_\theta(s, a)$, so the norm of the gradient is in addition clipped at 10.

For REINFORCE on the cart-pole each run has 600 episodes with one gradient step per batch of five episodes, the learning rate is 10^{-3} for both networks, the value network takes 20 regression steps per batch, and $\gamma = 0.99$. Each variant is trained with five random seeds.

References

[1] A. Dawid *et al.*, *Modern applications of machine learning in quantum sciences*, arXiv:2204.04198, chapter on reinforcement learning.

[2] J. Degrave *et al.*, Magnetic control of tokamak plasmas through deep reinforcement learning, Nature **602**, 414 (2022), doi:10.1038/s41586-021-04301-9.

[3] A. Fawzi *et al.*, Discovering faster matrix multiplication algorithms with reinforcement learning, Nature **610**, 47 (2022), doi:10.1038/s41586-022-05172-4.

[4] M. Komorowski, L. A. Celi, O. Badawi, A. C. Gordon and A. A. Faisal, The Artificial Intelligence Clinician learns optimal treatment strategies for sepsis in intensive care, Nat. Med. **24**, 1716 (2018), doi:10.1038/s41591-018-0213-5.

[5] M. Bukov and F. Marquardt, *Reinforcement learning for quantum technology*, arXiv:2601.18953 (2026).

[6] R. Bellman, *Dynamic Programming*, Princeton University Press (1957), reprinted 2010, doi:10.1515/9781400835386.

[7] F. Pardo, A. Tavakoli, V. Levdik and P. Kormushev, Time limits in reinforcement learning, Proc. ICML, PMLR **80**, 4042 (2018), arXiv:1712.00378.

[8] R. S. Sutton, Learning to predict by the methods of temporal differences, Mach. Learn. **3**, 9 (1988), doi:10.1007/BF00115009.

[9] T. Jaakkola, M. I. Jordan and S. P. Singh, On the convergence of stochastic iterative dynamic programming algorithms, Neural Comput. **6**, 1185 (1994), doi:10.1162/neco.1994.6.6.1185.

[10] J. N. Tsitsiklis, Asynchronous stochastic approximation and Q-learning, Mach. Learn. **16**, 185 (1994), doi:10.1007/BF00993306.

[11] H. Robbins and S. Monro, A stochastic approximation method, Ann. Math. Statist. **22**, 400 (1951), doi:10.1214/aoms/1177729586.

[12] P. Dayan, The convergence of TD(λ) for general λ, Mach. Learn. **8**, 341 (1992), doi:10.1007/BF00992701.

[13] C. J. C. H. Watkins and P. Dayan, Q-learning, Mach. Learn. **8**, 279 (1992), doi:10.1007/BF00992698.

[14] C. Nota and P. S. Thomas, Is the policy gradient a gradient?, AAMAS (2020), arXiv:1906.07073.

[15] R. J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, Mach. Learn. **8**, 229 (1992), doi:10.1007/BF00992696.

[16] M. Towers *et al.*, Gymnasium: a standard interface for reinforcement learning environments, arXiv:2407.17032 (2024).

[17] R. S. Sutton and A. G. Barto, *Reinforcement Learning. An Introduction*, 2nd ed., MIT Press (2018), `http://incompleteideas.net/book/the-book-2nd.html`.

[18] V. Mnih *et al.*, Human-level control through deep reinforcement learning, Nature **518**, 529 (2015), doi:10.1038/nature14236.

[19] H. van Hasselt, A. Guez and D. Silver, Deep reinforcement learning with double Q-learning, AAAI (2016), arXiv:1509.06461.

[20] A. G. Barto, R. S. Sutton and C. W. Anderson, Neuronlike adaptive elements that can solve difficult learning control problems, IEEE Trans. Syst. Man Cybern. **SMC-13**, 834 (1983), doi:10.1109/TSMC.1983.6313077.

[21] D. Silver *et al.*, Mastering the game of Go with deep neural networks and tree search, Nature **529**, 484 (2016), doi:10.1038/nature16961.

[22] D. Silver *et al.*, Mastering the game of Go without human knowledge, Nature **550**, 354 (2017), doi:10.1038/nature24270.

[23] D. Silver *et al.*, A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play, Science **362**, 1140 (2018), doi:10.1126/science.aar6404.

[24] J. Schrittwieser *et al.*, Mastering Atari, Go, chess and shogi by planning with a learned model, Nature **588**, 604 (2020), doi:10.1038/s41586-020-03051-4.

[25] V. Strassen, Gaussian elimination is not optimal, Numer. Math. **13**, 354 (1969), doi:10.1007/BF02165411.

[26] K. Nakaji *et al.*, Quantum circuits as a game, arXiv:2506.05536 (2025).

[27] F. Marquardt, Machine learning and quantum devices, SciPost Phys. Lect. Notes **29** (2021), doi:10.21468/SciPostPhysLectNotes.29.

[28] K. Reuer *et al.*, Realizing a deep reinforcement learning agent for real-time quantum feedback, Nat. Commun. **14**, 7138 (2023), doi:10.1038/s41467-023-42901-3.

[29] A. Paszke *et al.*, PyTorch: an imperative style, high-performance deep learning library, NeurIPS (2019), arXiv:1912.01703.

[30] D. P. Kingma and J. Ba, Adam: a method for stochastic optimization, ICLR (2015), arXiv:1412.6980.